PyTorch: Basic to Advanced

The Complete Visual Guide

by jw

Based on PyTorch 2.8.0 | 377 examples | 31 categories

Table of Contents

19. Optimizers & Schedulers (25 examples)
20. Regularization (7 examples)
21. Convolutions & Pooling (3 examples)
22. Recurrent Layers (2 examples)

Part V: Advanced

23. Attention & Transformers (6 examples)
24. Data Loading (3 examples)
25. Training Patterns (2 examples)
26. Custom Modules & Hooks (0 examples)
27. Mixed Precision (AMP) (1 examples)
28. Transforms (torchvision) (0 examples)
29. JIT & TorchScript (2 examples)
30. Profiling & Debugging (2 examples)
31. Distributed Training (0 examples)

Hello World

Your first neural network! We'll teach a model the XOR function.

This is the **complete** pattern you'll use for everything in deep learning.

Input A	Input B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

```
import torch
import torch.nn as nn

# Learn XOR: the "Hello World" of neural networks
X = torch.tensor([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
y = torch.tensor([[0.], [1.], [1.], [0.]])

model = nn.Sequential(nn.Linear(2, 8), nn.ReLU(), nn.Linear(8, 1), nn.Sigmoid())
optimizer = torch.optim.Adam(model.parameters(), lr=0.05)

print("Training...")
for epoch in range(500):
    loss = nn.BCELoss()(model(X), y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if epoch % 100 == 0:
        print(f"  Epoch {epoch:3d}  Loss: {loss.item():.4f}")

print("\nResults:")
```

```
  with torch.no_grad():
      pred = model(X)
      for i in range(4):
          print(f"  {X[i].tolist()} -> {pred[i].item():.3f} ({'1' if pred[i]>0.5 else '0'})")
  print(f"  Accuracy: {((pred>0.5).float() == y).float().mean().item()*100:.0f}%")
```

Output:

```
  Training...
    Epoch   0  Loss: 0.7061
    Epoch 100  Loss: 0.0023
    Epoch 200  Loss: 0.0007
    Epoch 300  Loss: 0.0003
    Epoch 400  Loss: 0.0002

  Results:
    [0.0, 0.0] -> 0.000 (0)
    [0.0, 1.0] -> 1.000 (1)
    [1.0, 0.0] -> 1.000 (1)
    [1.0, 1.0] -> 0.000 (0)
    Accuracy: 100%
```

Key pattern: DATA -> MODEL -> LOSS -> BACKWARD -> STEP -> repeat!

Part I: Foundations

!Tensor Creation Chart

1. Tensor Creation

20 functions with examples | 20 total in this category

torch.tensor(data)

Creates a tensor from Python data. Infers dtype automatically.

```
  import torch
  x = torch.tensor([[1, 2], [3, 4]])
  print(x)
  print(f"shape={x.shape}, dtype={x.dtype}")
```

```
  tensor([[1, 2],
          [3, 4]])
  shape=torch.Size([2, 2]), dtype=torch.int64
```

torch.zeros(*size)

All zeros. Great for initialization.

```
  import torch
```

```
print(torch.zeros(2, 3))
```

```
tensor([[0., 0., 0.],
        [0., 0., 0.]])
```

torch.ones(*size)

All ones. Useful for masks and broadcasting.

```
import torch
print(torch.ones(3, 2))
```

```
tensor([[1., 1.],
        [1., 1.],
        [1., 1.]])
```

torch.eye(n)

Identity matrix: 1s on diagonal, 0s elsewhere.

```
import torch
print(torch.eye(3))
```

```
tensor([[1., 0., 0.],
        [0., 1., 0.],
        [0., 0., 1.]])
```

torch.arange(start, end, step)

Like Python's range() but returns a tensor.

```
import torch
print(torch.arange(0, 10, 2))
print(torch.arange(5))
```

```
tensor([0, 2, 4, 6, 8])
tensor([0, 1, 2, 3, 4])
```

torch.linspace(start, end, steps)

N equally-spaced points between start and end (inclusive).

```
import torch
print(torch.linspace(0, 1, 5))
```

```
tensor([0.0000, 0.2500, 0.5000, 0.7500, 1.0000])
```

torch.logspace(start, end, steps)

Logarithmically spaced values. Handy for learning rates.

```
import torch
print(torch.logspace(0, 2, 3))  # 10^0, 10^1, 10^2
```

```
tensor([  1.,  10., 100.])
```

torch.rand(*size)

Uniform random in [0, 1). The workhorse of randomness.

```
import torch
torch.manual_seed(42)
print(torch.rand(2, 3))
```

```
tensor([[0.8823, 0.9150, 0.3829],
        [0.9593, 0.3904, 0.6009]])
```

torch.randn(*size)

Normal distribution N(0, 1). Used for weight initialization.

```
import torch
torch.manual_seed(42)
print(torch.randn(2, 3))
```

```
tensor([[ 0.3367,  0.1288,  0.2345],
        [ 0.2303, -1.1229, -0.1863]])
```

torch.randint(low, high, size)

Random integers in [low, high). Good for labels and indices.

```
import torch
torch.manual_seed(42)
print(torch.randint(0, 10, (2, 3)))
```

```
tensor([[2, 7, 6],
        [4, 6, 5]])
```

torch.randperm(n)

Random permutation of 0..n-1. Perfect for shuffling.

```
import torch
torch.manual_seed(42)
print(torch.randperm(8))
```

```
tensor([6, 3, 0, 7, 2, 1, 4, 5])
```

torch.empty(*size)

Uninitialized memory. Fast but dangerous if you forget to fill it.

```
import torch
x = torch.empty(2, 2)
print(x)
print("Warning: uninitialized garbage values!")
```

```
tensor([[0., 0.],
        [0., 0.]])
Warning: uninitialized garbage values!
```

torch.full(size, val)

Fill with any value you want.

```
import torch
print(torch.full((2, 3), 3.14))
```

```
tensor([[3.1400, 3.1400, 3.1400],
        [3.1400, 3.1400, 3.1400]])
```

torch.zeros_like(x)

Same shape and dtype as x, but all zeros.

```
import torch
x = torch.randn(2, 3)
print(torch.zeros_like(x))
```

```
tensor([[0., 0., 0.],
        [0., 0., 0.]])
```

torch.ones_like(x)

Same shape and dtype as x, but all ones.

```
import torch
x = torch.randn(2, 3)
print(torch.ones_like(x))
```

```
tensor([[1., 1., 1.],
        [1., 1., 1.]])
```

torch.rand_like(x)

Random values matching x's shape/dtype/device.

```
import torch
torch.manual_seed(42)
x = torch.zeros(2, 3)
print(torch.rand_like(x))
```

```
tensor([[0.8823, 0.9150, 0.3829],
        [0.9593, 0.3904, 0.6009]])
```

torch.complex(real, imag)

Create complex numbers. PyTorch has full complex support!

```
import torch
z = torch.complex(torch.tensor([1., 2.]), torch.tensor([3., 4.]))
print(f'{z}  real={z.real}  imag={z.imag}')
```

```
tensor([1.+3.j, 2.+4.j])  real=tensor([1., 2.])  imag=tensor([3., 4.])
```

torch.as_tensor(data)

Like tensor() but avoids copy if possible. Shares numpy memory.

```
import torch, numpy as np
arr = np.array([1, 2, 3])
t = torch.as_tensor(arr)
print(t)
```

```
tensor([1, 2, 3])
```

torch.from_numpy(arr)

Zero-copy conversion from numpy. Changes to one affect the other.

```
import torch, numpy as np
arr = np.array([1., 2., 3.])
t = torch.from_numpy(arr)
print(f'{t}  (shares memory with numpy!)')
```

```
tensor([1., 2., 3.], dtype=torch.float64)  (shares memory with numpy!)
```

torch.frombuffer(buf)

Create tensor from a buffer (bytes, array, etc.).

```
import torch, array
buf = array.array('f', [1.0, 2.0, 3.0])
t = torch.frombuffer(buf, dtype=torch.float32)
print(t)
```

```
tensor([1., 2., 3.])
```

!Tensor Properties Chart

2. Tensor Properties

14 functions with examples | 14 total in this category

x.shape / x.size()

Both give dimensions. shape is property, size() is method.

```
import torch
x = torch.randn(2, 3, 4)
print(f'shape: {x.shape}  size(): {x.size()}')
```

```
shape: torch.Size([2, 3, 4])  size(): torch.Size([2, 3, 4])
```

x.dtype

Data type. Int defaults to int64, float to float32.

```
import torch
print(torch.tensor([1]).dtype)
print(torch.tensor([1.0]).dtype)
```

```
torch.int64
torch.float32
```

x.device

Where the tensor lives: cpu, cuda:0, mps, etc.

```
import torch
print(torch.tensor([1]).device)
```

```
cpu
```

x.dim()

Number of dimensions (rank). Scalar=0, Vector=1, Matrix=2.

```
import torch
print(f'scalar: {torch.tensor(5).dim()}D')
print(f'vector: {torch.tensor([1,2,3]).dim()}D')
print(f'matrix: {torch.randn(2,3).dim()}D')
```

```
scalar: 0D
vector: 1D
matrix: 2D
```

x.numel()

Total element count. Product of all dimensions.

```
import torch
x = torch.randn(2, 3, 4)
print(f'{x.shape} -> {x.numel()} elements')
```

```
torch.Size([2, 3, 4]) -> 24 elements
```

x.is_contiguous()

Is memory layout C-contiguous? Transpose breaks it.

```
import torch
x = torch.randn(3, 4)
print(f'original: {x.is_contiguous()}')
print(f'transposed: {x.T.is_contiguous()}')
```

```
original: True
transposed: False
```

x.is_cuda

Quick check if tensor is on GPU.

```
import torch
x = torch.tensor([1.])
print(f'is_cuda: {x.is_cuda}')
```

```
is_cuda: False
```

x.is_floating_point()

True for float16, float32, float64, bfloat16.

```
import torch
print(f'float: {torch.tensor([1.]).is_floating_point()}')
print(f'int:   {torch.tensor([1]).is_floating_point()}')
```

```
float: True
int:   False
```

x.requires_grad

Is autograd tracking this tensor?

```
import torch
x = torch.tensor([1.], requires_grad=True)
print(f'requires_grad: {x.requires_grad}')
```

```
requires_grad: True
```

x.grad

Gradient stored after backward(). Accumulates!

```
import torch
x = torch.tensor(3., requires_grad=True)
(x ** 3).backward()
print(f'x={x}, d(x^3)/dx = 3x^2 = {x.grad}')
```

```
x=3.0, d(x^3)/dx = 3x^2 = 27.0
```

x.data

Raw tensor without autograd. Use with caution!

```
import torch
x = torch.tensor([1.], requires_grad=True)
print(f'x.data: {x.data}  (no grad tracking)')
```

```
x.data: tensor([1.])  (no grad tracking)
```

x.T

Transpose shortcut for 2D tensors.

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print(f'x:   {x.shape}\n{x}')
print(f'x.T: {x.T.shape}\n{x.T}')
```

```
x:   torch.Size([2, 3])
tensor([[1, 2, 3],
        [4, 5, 6]])
x.T: torch.Size([3, 2])
tensor([[1, 4],
        [2, 5],
        [3, 6]])
```

x.real / x.imag

Access real and imaginary parts of complex tensors.

```
import torch
z = torch.tensor([1+2j, 3+4j])
print(f'real: {z.real}, imag: {z.imag}')
```

```
real: tensor([1., 3.]), imag: tensor([2., 4.])
```

x.nbytes

Memory consumed by tensor data.

```
import torch
x = torch.randn(1000, 1000)
print(f'{x.shape} float32 = {x.nbytes / 1e6:.1f} MB')
```

```
torch.Size([1000, 1000]) float32 = 4.0 MB
```

!Dtype Conversion Chart

3. Dtype Conversion

10 functions with examples | 10 total in this category

x.float()

Convert to float32. Most common conversion.

```
import torch
x = torch.tensor([1, 2, 3])
print(f'{x.dtype} -> {x.float().dtype}')
```

```
torch.int64 -> torch.float32
```

x.double()

Convert to float64. Higher precision, 2x memory.

```
import torch
print(torch.tensor([1.]).double().dtype)
```

```
torch.float64
```

x.half()

Convert to float16. Half memory, good for inference.

```
import torch
print(torch.tensor([1.]).half().dtype)
```

```
torch.float16
```

x.bfloat16()

Brain float16. Same range as float32, less precision. Training-friendly.

```
import torch
print(torch.tensor([1.]).bfloat16().dtype)
```

```
torch.bfloat16
```

x.int()

Convert to int32. Truncates decimals.

```
import torch
print(torch.tensor([1.7, 2.3]).int())
```

```
tensor([1, 2], dtype=torch.int32)
```

x.long()

Convert to int64. Required for indices and labels.

```
import torch
print(torch.tensor([1.5]).long().dtype)
```

```
torch.int64
```

x.short()

Convert to int16.

```
import torch
print(torch.tensor([1]).short().dtype)
```

```
torch.int16
```

x.bool()

Convert to bool. 0->False, everything else->True.

```
import torch
print(torch.tensor([0, 1, -1, 0.5]).bool())
```

```
tensor([False,  True,  True,  True])
```

x.to(dtype)

General-purpose dtype conversion.

```
import torch
x = torch.tensor([1, 2])
print(x.to(torch.float16))
```

```
tensor([1., 2.], dtype=torch.float16)
```

x.type(dtype)

Convert using type name string or class.

```
import torch
x = torch.tensor([1.0])
print(x.type(torch.LongTensor).dtype)
```

```
torch.int64
```

!Tensor Shapes Chart

4. Shape Manipulation

16 functions with examples | 16 total in this category

x.reshape(*shape)

New shape, same data. -1 means 'figure it out'.

```
import torch
x = torch.arange(12)
print(f'{x} ->\n{x.reshape(3, 4)}')
```

```
tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11]) ->
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

x.view(*shape)

Like reshape but requires contiguous memory. Faster.

```
import torch
x = torch.arange(12)
print(x.view(3, 4))
print(x.view(2, -1))  # -1 = auto
```

```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
tensor([[ 0,  1,  2,  3,  4,  5],
        [ 6,  7,  8,  9, 10, 11]])
```

x.transpose(d0, d1)

Swap two dimensions.

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print(f'{x.shape} -> {x.transpose(0,1).shape}')
```

```
torch.Size([2, 3]) -> torch.Size([3, 2])
```

x.permute(*dims)

Reorder ALL dimensions at once.

```
import torch
x = torch.randn(2, 3, 4)  # batch, height, width
print(f'{x.shape} -> {x.permute(0, 2, 1).shape}  # swap H,W')
```

```
torch.Size([2, 3, 4]) -> torch.Size([2, 4, 3])  # swap H,W
```

x.squeeze(dim)

Remove dimensions of size 1. squeeze(dim) is safer.

```
import torch
x = torch.randn(1, 3, 1, 4)
print(f'{x.shape} -> squeeze: {x.squeeze().shape}')
print(f'{x.shape} -> squeeze(0): {x.squeeze(0).shape}')
```

```
torch.Size([1, 3, 1, 4]) -> squeeze: torch.Size([3, 4])
torch.Size([1, 3, 1, 4]) -> squeeze(0): torch.Size([3, 1, 4])
```

x.unsqueeze(dim)

Add a dimension of size 1. Essential for broadcasting.

```
import torch
x = torch.tensor([1, 2, 3])
print(f'{x.shape} -> unsqueeze(0): {x.unsqueeze(0).shape}  # add batch dim')
print(f'{x.shape} -> unsqueeze(1): {x.unsqueeze(1).shape}  # add feature dim')
```

```
torch.Size([3]) -> unsqueeze(0): torch.Size([1, 3])  # add batch dim
torch.Size([3]) -> unsqueeze(1): torch.Size([3, 1])  # add feature dim
```

x.flatten(start_dim)

Collapse dimensions. flatten(1) is your CNN->FC friend.

```
import torch
x = torch.randn(2, 3, 4)
print(f'flatten():  {x.flatten().shape}')
print(f'flatten(1): {x.flatten(1).shape}  # keep batch dim')
```

```
flatten():  torch.Size([24])
flatten(1): torch.Size([2, 12])  # keep batch dim
```

x.unflatten(dim, sizes)

Reverse of flatten. Split a dim into multiple.

```
import torch
x = torch.randn(2, 12)
print(f'{x.shape} -> {x.unflatten(1, (3, 4)).shape}')
```

```
torch.Size([2, 12]) -> torch.Size([2, 3, 4])
```

x.expand(*sizes)

Broadcast without copying data. Memory efficient!

```
import torch
x = torch.tensor([[1], [2], [3]])
print(f'{x.shape} -> expand(3,4):\n{x.expand(3, 4)}')
```

```
torch.Size([3, 1]) -> expand(3,4):
tensor([[1, 1, 1, 1],
        [2, 2, 2, 2],
        [3, 3, 3, 3]])
```

x.expand_as(other)

Expand to match another tensor's shape.

```
import torch
x = torch.tensor([1, 2, 3])
y = torch.randn(2, 3)
print(f'{x.shape} -> {x.expand_as(y).shape}')
```

```
torch.Size([3]) -> torch.Size([2, 3])
```

x.repeat(*sizes)

Actually copies data. Use expand when possible.

```
import torch
x = torch.tensor([1, 2, 3])
print(x.repeat(2, 3))
```

```
tensor([[1, 2, 3, 1, 2, 3, 1, 2, 3],
        [1, 2, 3, 1, 2, 3, 1, 2, 3]])
```

x.repeat_interleave(repeats)

Repeat each element. Like numpy.repeat.

```
import torch
x = torch.tensor([1, 2, 3])
print(x.repeat_interleave(2))
```

```
tensor([1, 1, 2, 2, 3, 3])
```

x.contiguous()

Make memory layout contiguous. Required before view().

```
import torch
x = torch.randn(3, 4).T  # T makes it non-contiguous
print(f'contiguous: {x.is_contiguous()} -> {x.contiguous().is_contiguous()}')
```

```
contiguous: False -> True
```

x.narrow(dim, start, len)

Slice along a dimension. Like x[2:7] but functional.

```
import torch
x = torch.arange(10)
print(f'narrow(0, 2, 5): {x.narrow(0, 2, 5)}')
```

```
narrow(0, 2, 5): tensor([2, 3, 4, 5, 6])
```

x.movedim(src, dst)

Move a dimension to a new position.

```
import torch
x = torch.randn(2, 3, 4)
print(f'{x.shape} -> {x.movedim(0, 2).shape}')
```

```
torch.Size([2, 3, 4]) -> torch.Size([3, 4, 2])
```

x.tile(dims)

Tile/repeat tensor. Like numpy.tile.

```
import torch
x = torch.tensor([1, 2, 3])
print(x.tile((2, 3)))
```

```
tensor([[1, 2, 3, 1, 2, 3, 1, 2, 3],
        [1, 2, 3, 1, 2, 3, 1, 2, 3]])
```

!Indexing Chart

5. Indexing & Slicing

15 functions with examples | 15 total in this category

x[i], x[i:j], x[..., i]

Standard Python indexing works on tensors.

```
import torch
x = torch.arange(12).reshape(3, 4)
print(f"x:\n{x}")
print(f"x[1]:       {x[1]}")
print(f"x[:, -1]:   {x[:, -1]}")
print(f"x[0:2, 1:]: \n{x[0:2, 1:]}")
```

```
x:
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
x[1]:       tensor([4, 5, 6, 7])
x[:, -1]:   tensor([ 3,  7, 11])
```

```
x[0:2, 1:]:
tensor([[1, 2, 3],
        [5, 6, 7]])
```

x[mask] # boolean indexing

Filter elements with a boolean mask. Returns 1D tensor.

```
import torch
x = torch.tensor([1, 2, 3, 4, 5])
mask = x > 3
print(f'x[x > 3] = {x[mask]}')
```

```
x[x > 3] = tensor([4, 5])
```

torch.index_select(x, dim, idx)

Select specific indices along a dimension.

```
import torch
x = torch.arange(12).reshape(3, 4)
idx = torch.tensor([0, 2])
print(f'Select rows 0,2:\n{torch.index_select(x, 0, idx)}')
```

```
Select rows 0,2:
tensor([[ 0,  1,  2,  3],
        [ 8,  9, 10, 11]])
```

torch.masked_select(x, mask)

Returns flat tensor of elements where mask=True.

```
import torch
x = torch.tensor([[1, 2], [3, 4]])
mask = x > 2
print(f'masked_select: {torch.masked_select(x, mask)}')
```

```
masked_select: tensor([3, 4])
```

torch.gather(x, dim, idx)

Collect values along dim using index. Key for RL reward gathering.

```
import torch
x = torch.tensor([[1, 2], [3, 4]])
idx = torch.tensor([[0, 0], [1, 0]])
print(f'gather(dim=1):\n{torch.gather(x, 1, idx)}')
```

```
gather(dim=1):
tensor([[1, 1],
        [4, 3]])
```

x.scatter_(dim, idx, src)

Scatter values into positions. Perfect for one-hot encoding!

```
import torch
x = torch.zeros(3, 5, dtype=torch.long)
idx = torch.tensor([[0], [2], [4]])
x.scatter_(1, idx, 1)
print(f'One-hot:\n{x}')
```

```
One-hot:
tensor([[1, 0, 0, 0, 0],
        [0, 0, 1, 0, 0],
        [0, 0, 0, 0, 1]])
```

torch.where(cond, x, y)

If-else on tensors. Pick from x where True, y where False.

```
import torch
x = torch.tensor([1, 2, 3, 4, 5])
print(f'where(x>3, x, 0): {torch.where(x > 3, x, torch.tensor(0))}')
```

```
where(x>3, x, 0): tensor([0, 0, 0, 4, 5])
```

torch.take(x, indices)

Index into flattened tensor.

```
import torch
x = torch.tensor([[1, 2], [3, 4]])
print(f'take flat idx [0,3]: {torch.take(x, torch.tensor([0, 3]))}')
```

```
take flat idx [0,3]: tensor([1, 4])
```

torch.take_along_dim(x, idx, dim)

Gather along a specific dimension.

```
import torch
x = torch.tensor([[10, 20, 30], [40, 50, 60]])
idx = torch.tensor([[2], [0]])
print(torch.take_along_dim(x, idx, dim=1))
```

```
tensor([[30],
        [40]])
```

torch.nonzero(x)

Find indices of non-zero elements.

```
import torch
x = torch.tensor([0, 1, 0, 2, 0, 3])
```

```
print(f'nonzero indices: {torch.nonzero(x).squeeze()}')
```

```
nonzero indices: tensor([1, 3, 5])
```

torch.triu(x, diagonal)

Upper triangle. triu with diagonal=-1 for causal masks!

```
import torch
x = torch.ones(4, 4)
print(f'Upper triangular:\n{torch.triu(x)}')
```

```
Upper triangular:
tensor([[1., 1., 1., 1.],
        [0., 1., 1., 1.],
        [0., 0., 1., 1.],
        [0., 0., 0., 1.]])
```

torch.tril(x, diagonal)

Lower triangle. Used for causal attention masks.

```
import torch
print(f'Lower triangular:\n{torch.tril(torch.ones(4, 4))}')
```

```
Lower triangular:
tensor([[1., 0., 0., 0.],
        [1., 1., 0., 0.],
        [1., 1., 1., 0.],
        [1., 1., 1., 1.]])
```

torch.diagonal(x)

Extract diagonal elements.

```
import torch
x = torch.arange(9).reshape(3, 3)
print(f'diagonal: {torch.diagonal(x)}')
```

```
diagonal: tensor([0, 4, 8])
```

torch.diag(x)

Create diagonal matrix from 1D, or extract diagonal from 2D.

```
import torch
print(f'diag from vector:\n{torch.diag(torch.tensor([1, 2, 3]))}')
```

```
diag from vector:
tensor([[1, 0, 0],
        [0, 2, 0],
        [0, 0, 3]])
```

torch.index_put_(x, indices, values)

Put values at specific indices. In-place.

```
import torch
x = torch.zeros(5)
idx = (torch.tensor([0, 2, 4]),)
x.index_put_(idx, torch.tensor([10., 20., 30.]))
print(x)
```

```
tensor([10.,  0., 20.,  0., 30.])
```

!Join Split Chart

6. Joining & Splitting

11 functions with examples | 11 total in this category

torch.cat(tensors, dim)

Concatenate along existing dimension. Shapes must match except in that dim.

```
import torch
a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[5, 6]])
print(f'cat(dim=0):\n{torch.cat([a, b], dim=0)}')
```

```
cat(dim=0):
tensor([[1, 2],
        [3, 4],
        [5, 6]])
```

torch.stack(tensors, dim)

Stack along NEW dimension. All tensors must have same shape.

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
print(f'stack(dim=0):\n{torch.stack([a, b], dim=0)}')
```

```
stack(dim=0):
tensor([[1, 2, 3],
        [4, 5, 6]])
```

torch.vstack(tensors)

Stack vertically (row-wise). Like cat(dim=0) for 1D inputs.

```
import torch
a = torch.tensor([1, 2])
b = torch.tensor([3, 4])
print(torch.vstack([a, b]))
```

```
tensor([[1, 2],
        [3, 4]])
```

torch.hstack(tensors)

Stack horizontally. Like cat(dim=1) for 2D.

```
import torch
a = torch.tensor([1, 2])
b = torch.tensor([3, 4])
print(torch.hstack([a, b]))
```

```
tensor([1, 2, 3, 4])
```

torch.dstack(tensors)

Stack along third dimension (depth).

```
import torch
a = torch.tensor([1, 2])
b = torch.tensor([3, 4])
print(torch.dstack([a, b]).shape)
```

```
torch.Size([1, 2, 2])
```

torch.split(x, size, dim)

Split into chunks of given size. Last may be smaller.

```
import torch
x = torch.arange(10)
for i, s in enumerate(torch.split(x, 3)):
    print(f'  chunk {i}: {s}')
```

```
chunk 0: tensor([0, 1, 2])
  chunk 1: tensor([3, 4, 5])
  chunk 2: tensor([6, 7, 8])
  chunk 3: tensor([9])
```

torch.chunk(x, chunks, dim)

Split into N equal-ish chunks.

```
import torch
for i, c in enumerate(torch.chunk(torch.arange(12), 4)):
    print(f'  chunk {i}: {c}')
```

```
chunk 0: tensor([0, 1, 2])
  chunk 1: tensor([3, 4, 5])
```

```
  chunk 2: tensor([6, 7, 8])
  chunk 3: tensor([ 9, 10, 11])
```

torch.tensor_split(x, sections)

Split at specific indices. More flexible than split/chunk.

```
import torch
for i, s in enumerate(torch.tensor_split(torch.arange(10), [3, 7])):
    print(f'  {i}: {s}')
```

```
0: tensor([0, 1, 2])
  1: tensor([3, 4, 5, 6])
  2: tensor([7, 8, 9])
```

torch.unbind(x, dim)

Remove a dimension and return tuple of slices.

```
import torch
x = torch.tensor([[1, 2], [3, 4], [5, 6]])
for row in torch.unbind(x, dim=0):
    print(f'  {row}')
```

```
tensor([1, 2])
  tensor([3, 4])
  tensor([5, 6])
```

torch.column_stack(tensors)

Stack 1D tensors as columns of a 2D tensor.

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
print(torch.column_stack([a, b]))
```

```
tensor([[1, 4],
        [2, 5],
        [3, 6]])
```

torch.row_stack(tensors)

Stack 1D tensors as rows. Same as vstack.

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
print(torch.row_stack([a, b]))
```

```
tensor([[1, 2, 3],
        [4, 5, 6]])
```

Part II: Math

!Pointwise Math Chart

7. Math -- Pointwise

46 functions with examples | 46 total in this category

torch.add(a, b)

a + b. Supports broadcasting and alpha: add(a, b, alpha=2) = a + 2b.*

```
import torch
print(torch.add(torch.tensor([1, 2]), torch.tensor([3, 4])))
```

```
tensor([4, 6])
```

torch.sub(a, b)

a - b element-wise.

```
import torch
print(torch.sub(torch.tensor([5, 6]), torch.tensor([1, 2])))
```

```
tensor([4, 4])
```

torch.mul(a, b)

a b element-wise (Hadamard product). NOT matrix multiply!*

```
import torch
print(torch.mul(torch.tensor([2, 3]), torch.tensor([4, 5])))
```

```
tensor([ 8, 15])
```

torch.div(a, b)

a / b element-wise.

```
import torch
print(torch.div(torch.tensor([10., 20.]), torch.tensor([3., 7.])))
```

```
tensor([3.3333, 2.8571])
```

torch.pow(a, exp)

a^exp. Also: a exp.

```
import torch
print(torch.pow(torch.tensor([1., 2., 3.]), 2))
```

```
tensor([1., 4., 9.])
```

torch.sqrt(x)

Square root. For rsqrt: 1/sqrt(x).

```
import torch
print(torch.sqrt(torch.tensor([1., 4., 9., 16.])))
```

```
tensor([1., 2., 3., 4.])
```

torch.rsqrt(x)

1/sqrt(x). Faster than 1/torch.sqrt(x) on GPU.

```
import torch
print(torch.rsqrt(torch.tensor([1., 4., 9.])))
```

```
tensor([1.0000, 0.5000, 0.3333])
```

torch.abs(x)

Absolute value.

```
import torch
print(torch.abs(torch.tensor([-3, -1, 0, 2])))
```

```
tensor([3, 1, 0, 2])
```

torch.neg(x)

Negate: -x.

```
import torch
print(torch.neg(torch.tensor([1, -2, 3])))
```

```
tensor([-1,  2, -3])
```

torch.sign(x)

Returns -1, 0, or 1.

```
import torch
print(torch.sign(torch.tensor([-5., 0., 3.])))
```

```
tensor([-1.,  0.,  1.])
```

torch.ceil(x)

Round up to nearest integer.

```
import torch
print(torch.ceil(torch.tensor([-1.7, 0.3, 1.2])))
```

```
tensor([-1.,  1.,  2.])
```

torch.floor(x)

Round down to nearest integer.

```
import torch
print(torch.floor(torch.tensor([-1.7, 0.3, 1.8])))
```

```
tensor([-2.,  0.,  1.])
```

torch.round(x)

Round to nearest even integer (banker's rounding).

```
import torch
print(torch.round(torch.tensor([-1.7, 0.3, 1.5, 2.5])))
```

```
tensor([-2.,  0.,  2.,  2.])
```

torch.trunc(x)

Truncate toward zero.

```
import torch
print(torch.trunc(torch.tensor([-1.7, 0.3, 1.8])))
```

```
tensor([-1.,  0.,  1.])
```

torch.frac(x)

Fractional part: x - trunc(x).

```
import torch
print(torch.frac(torch.tensor([1.7, -2.3, 3.0])))
```

```
tensor([ 0.7000, -0.3000,  0.0000])
```

torch.clamp(x, min, max)

Clip values to [min, max]. Your gradient clipping friend.

```
import torch
```

```
x = torch.tensor([-2., -1., 0., 1., 2.])
print(f'clamp(-1, 1): {torch.clamp(x, -1, 1)}')
```

```
clamp(-1, 1): tensor([-1., -1.,  0.,  1.,  1.])
```

torch.clip(x, min, max)

Alias for clamp. Same thing, numpy-style name.

```
import torch
print(torch.clip(torch.tensor([-5., 0., 5.]), -1, 1))
```

```
tensor([-1.,  0.,  1.])
```

torch.remainder(a, b)

Modulo. Result has same sign as divisor (Python-style).

```
import torch
print(torch.remainder(torch.tensor([7, -7]), torch.tensor([3, 3])))
```

```
tensor([1, 2])
```

torch.fmod(a, b)

Modulo. Result has same sign as dividend (C-style).

```
import torch
print(torch.fmod(torch.tensor([7, -7]), torch.tensor([3, 3])))
```

```
tensor([ 1, -1])
```

torch.floor_divide(a, b)

Integer division: a // b.

```
import torch
print(torch.floor_divide(torch.tensor([7., -7.]), torch.tensor([2., 2.])))
```

```
tensor([ 3., -4.])
```

torch.reciprocal(x)

1/x for each element.

```
import torch
print(torch.reciprocal(torch.tensor([1., 2., 4.])))
```

```
tensor([1.0000, 0.5000, 0.2500])
```

torch.exp(x)

e^x. The foundation of softmax.

```
import torch
print(torch.exp(torch.tensor([0., 1., 2.])))
```

```
tensor([1.0000, 2.7183, 7.3891])
```

torch.exp2(x)

2^x.

```
import torch
print(torch.exp2(torch.tensor([0., 1., 3.])))
```

```
tensor([1., 2., 8.])
```

torch.expm1(x)

e^x - 1. More accurate for small x.

```
import torch
print(torch.expm1(torch.tensor([0., 0.01])))
```

```
tensor([0.0000, 0.0101])
```

torch.log(x)

Natural logarithm (base e).

```
import torch
print(torch.log(torch.tensor([1., 2.718, 7.389])))
```

```
tensor([0.0000, 0.9999, 2.0000])
```

torch.log2(x)

Logarithm base 2.

```
import torch
print(torch.log2(torch.tensor([1., 2., 4., 8.])))
```

```
tensor([0., 1., 2., 3.])
```

torch.log10(x)

Logarithm base 10.

```
import torch
```

```
print(torch.log10(torch.tensor([1., 10., 100.])))
```

```
tensor([0., 1., 2.])
```

torch.log1p(x)

log(1 + x). Numerically stable for small x.

```
import torch
print(torch.log1p(torch.tensor([0., 0.01, 1.])))
```

```
tensor([0.0000, 0.0100, 0.6931])
```

torch.sin(x)

Sine (radians).

```
import torch, math
print(torch.sin(torch.tensor([0., math.pi/2, math.pi])))
```

```
tensor([ 0.0000e+00,  1.0000e+00, -8.7423e-08])
```

torch.cos(x)

Cosine (radians).

```
import torch, math
print(torch.cos(torch.tensor([0., math.pi/2, math.pi])))
```

```
tensor([ 1.0000e+00, -4.3711e-08, -1.0000e+00])
```

torch.tan(x)

Tangent (radians).

```
import torch, math
print(torch.tan(torch.tensor([0., math.pi/4])))
```

```
tensor([0.0000, 1.0000])
```

torch.asin(x)

Inverse sine (arcsin).

```
import torch
print(torch.asin(torch.tensor([0., 0.5, 1.])))
```

```
tensor([0.0000, 0.5236, 1.5708])
```

torch.acos(x)

Inverse cosine (arccos).

```
import torch
print(torch.acos(torch.tensor([0., 0.5, 1.])))
```

```
tensor([1.5708, 1.0472, 0.0000])
```

torch.atan(x)

Inverse tangent (arctan).

```
import torch
print(torch.atan(torch.tensor([0., 1., 100.])))
```

```
tensor([0.0000, 0.7854, 1.5608])
```

torch.atan2(y, x)

Two-argument arctan. Handles quadrant correctly.

```
import torch
print(torch.atan2(torch.tensor([1., 0.]), torch.tensor([0., 1.])))
```

```
tensor([1.5708, 0.0000])
```

torch.sinh(x)

Hyperbolic sine.

```
import torch
print(torch.sinh(torch.tensor([0., 1.])))
```

```
tensor([0.0000, 1.1752])
```

torch.cosh(x)

Hyperbolic cosine.

```
import torch
print(torch.cosh(torch.tensor([0., 1.])))
```

```
tensor([1.0000, 1.5431])
```

torch.tanh(x)

Hyperbolic tangent. Maps to (-1, 1).

```
import torch
```

```
print(torch.tanh(torch.tensor([-2., 0., 2.])))
```

```
tensor([-0.9640,  0.0000,  0.9640])
```

torch.sigmoid(x)

1/(1+e^-x). Maps to (0, 1).

```
import torch
print(torch.sigmoid(torch.tensor([-2., 0., 2.])))
```

```
tensor([0.1192, 0.5000, 0.8808])
```

torch.logit(x)

Inverse sigmoid: log(x/(1-x)).

```
import torch
print(torch.logit(torch.tensor([0.1, 0.5, 0.9])))
```

```
tensor([-2.1972,  0.0000,  2.1972])
```

torch.erf(x)

Error function. Used in GELU activation.

```
import torch
print(torch.erf(torch.tensor([0., 0.5, 1., 2.])))
```

```
tensor([0.0000, 0.5205, 0.8427, 0.9953])
```

torch.erfc(x)

Complementary error function: 1 - erf(x).

```
import torch
print(torch.erfc(torch.tensor([0., 1., 2.])))
```

```
tensor([1.0000, 0.1573, 0.0047])
```

torch.erfinv(x)

Inverse error function.

```
import torch
print(torch.erfinv(torch.tensor([0., 0.5, -0.5])))
```

```
tensor([ 0.0000,  0.4769, -0.4769])
```

torch.lerp(start, end, weight)

Linear interpolation: start + weight (end - start).*

```
import torch
a = torch.tensor([0., 10.])
b = torch.tensor([10., 20.])
print(f'lerp(0.3): {torch.lerp(a, b, 0.3)}')
```

```
lerp(0.3): tensor([ 3., 13.])
```

torch.addcmul(t, t1, t2, value)

t + value t1 t2. Fused operation.

```
import torch
t = torch.tensor([1., 2.])
t1 = torch.tensor([3., 4.])
t2 = torch.tensor([5., 6.])
print(torch.addcmul(t, t1, t2, value=0.1))
```

```
tensor([2.5000, 4.4000])
```

torch.addcdiv(t, t1, t2, value)

t + value t1 / t2. Used in optimizers internally.*

```
import torch
t = torch.tensor([1., 2.])
t1 = torch.tensor([3., 4.])
t2 = torch.tensor([5., 2.])
print(torch.addcdiv(t, t1, t2, value=0.1))
```

```
tensor([1.0600, 2.2000])
```

!Reduction Chart

8. Math -- Reduction

26 functions with examples | 26 total in this category

x.sum(dim)

Sum elements. dim=0: across rows, dim=1: across columns.

```
import torch
x = torch.tensor([[1., 2.], [3., 4.]])
print(f'sum():  {x.sum()}')
print(f'sum(0): {x.sum(0)}')
```

```
print(f'sum(1): {x.sum(1)}')
```

```
sum():  10.0
sum(0): tensor([4., 6.])
sum(1): tensor([3., 7.])
```

x.mean(dim)

Average. Requires float dtype!

```
import torch
x = torch.tensor([[1., 2.], [3., 4.]])
print(f'mean(): {x.mean()}, mean(0): {x.mean(0)}')
```

```
mean(): 2.5, mean(0): tensor([2., 3.])
```

x.std(dim)

Standard deviation (Bessel-corrected by default).

```
import torch
x = torch.tensor([2., 4., 4., 4., 5., 5., 7., 9.])
print(f'std: {x.std():.4f}')
```

```
std: 2.1381
```

x.var(dim)

Variance.

```
import torch
x = torch.tensor([2., 4., 4., 4., 5., 5., 7., 9.])
print(f'var: {x.var():.4f}')
```

```
var: 4.5714
```

x.prod(dim)

Product of all elements (1234 = 24).*

```
import torch
print(torch.tensor([1, 2, 3, 4]).prod())
```

```
tensor(24)
```

x.max(dim)

Max value + index. Without dim: global max.

```
import torch
x = torch.tensor([[1, 5], [7, 2]])
```

```
vals, idxs = x.max(dim=1)
print(f'max values: {vals}, indices: {idxs}')
```

```
max values: tensor([5, 7]), indices: tensor([1, 0])
```

x.min(dim)

Min value + index.

```
import torch
vals, idxs = torch.tensor([[3, 1], [2, 4]]).min(dim=1)
print(f'min values: {vals}, indices: {idxs}')
```

```
min values: tensor([1, 2]), indices: tensor([1, 0])
```

x.amax(dim)

Max along dim. Returns values only (no indices). Supports multiple dims.

```
import torch
x = torch.tensor([[1, 5], [7, 2]])
print(f'amax(0): {x.amax(0)}, amax(1): {x.amax(1)}')
```

```
amax(0): tensor([7, 5]), amax(1): tensor([5, 7])
```

x.amin(dim)

Min along dim. Values only.

```
import torch
print(torch.tensor([[1, 5], [7, 2]]).amin(0))
```

```
tensor([1, 2])
```

x.argmax(dim)

Index of max value. Crucial for classification predictions.

```
import torch
x = torch.tensor([[1, 5], [7, 2]])
print(f'argmax(1): {x.argmax(1)}')
```

```
argmax(1): tensor([1, 0])
```

x.argmin(dim)

Index of min value.

```
import torch
print(torch.tensor([3, 1, 2]).argmin())
```

```
tensor(1)
```

x.median(dim)

Median value.

```
import torch
print(torch.tensor([1., 3., 2., 5., 4.]).median())
```

```
tensor(3.)
```

x.nanmedian(dim)

Median ignoring NaN values.

```
import torch
print(torch.tensor([1., float('nan'), 3.]).nanmedian())
```

```
tensor(1.)
```

x.mode(dim)

Most frequent value.

```
import torch
vals, idxs = torch.tensor([1, 2, 2, 3, 3, 3]).mode(0)
print(f'mode: {vals}, index: {idxs}')
```

```
mode: 3, index: 5
```

x.quantile(q, dim)

Quantile/percentile values.

```
import torch
x = torch.arange(10).float()
print(f'25th percentile: {x.quantile(0.25)}')
print(f'75th percentile: {x.quantile(0.75)}')
```

```
25th percentile: 2.25
75th percentile: 6.75
```

x.cumsum(dim)

Running total: [1, 1+2, 1+2+3, ...].

```
import torch
x = torch.tensor([1, 2, 3, 4])
print(f'{x} -> cumsum: {x.cumsum(0)}')
```

```
tensor([1, 2, 3, 4]) -> cumsum: tensor([ 1,  3,  6, 10])
```

x.cumprod(dim)

Running product: [1, 12, 123, ...].*

```
import torch
print(torch.tensor([1, 2, 3, 4]).cumprod(0))
```

```
tensor([ 1,  2,  6, 24])
```

x.cummax(dim)

Running maximum.

```
import torch
vals, idxs = torch.tensor([3, 1, 4, 1, 5]).cummax(0)
print(f'cummax: {vals}')
```

```
cummax: tensor([3, 3, 4, 4, 5])
```

x.cummin(dim)

Running minimum.

```
import torch
vals, idxs = torch.tensor([3, 1, 4, 1, 5]).cummin(0)
print(f'cummin: {vals}')
```

```
cummin: tensor([3, 1, 1, 1, 1])
```

x.logcumsumexp(dim)

Numerically stable log(cumsum(exp(x))).

```
import torch
x = torch.tensor([1., 2., 3.])
print(f'logcumsumexp: {x.logcumsumexp(0)}')
```

```
logcumsumexp: tensor([1.0000, 2.3133, 3.4076])
```

torch.norm(x, p, dim)

Vector norm. L2=Euclidean, L1=Manhattan.

```
import torch
x = torch.tensor([3., 4.])
print(f'L2: {torch.norm(x, 2)}, L1: {torch.norm(x, 1)}')
```

```
L2: 5.0, L1: 7.0
```

torch.dist(a, b, p)

Lp distance between two tensors.

```
import torch
a = torch.tensor([1., 2.])
b = torch.tensor([4., 6.])
print(f'distance: {torch.dist(a, b, 2)}')
```

```
distance: 5.0
```

x.count_nonzero(dim)

Count non-zero elements.

```
import torch
x = torch.tensor([0, 1, 0, 3, 0])
print(f'nonzero count: {x.count_nonzero()}')
```

```
nonzero count: 2
```

torch.logsumexp(x, dim)

log(sum(exp(x))). Numerically stable. Used in softmax.

```
import torch
x = torch.tensor([1., 2., 3.])
print(f'logsumexp: {torch.logsumexp(x, 0):.4f}')
```

```
logsumexp: 3.4076
```

x.all(dim)

True if ALL elements are True.

```
import torch
print(torch.tensor([True, True, False]).all())
```

```
tensor(False)
```

x.any(dim)

True if ANY element is True.

```
import torch
print(torch.tensor([True, False, False]).any())
```

```
tensor(True)
```

!Comparison Chart

9. Math -- Comparison

23 functions with examples | 23 total in this category

torch.eq(a, b)

Element-wise ==. Returns bool tensor.

```
import torch
print(torch.eq(torch.tensor([1, 2, 3]), torch.tensor([1, 0, 3])))
```

```
tensor([ True, False,  True])
```

torch.ne(a, b)

Element-wise !=.

```
import torch
print(torch.ne(torch.tensor([1, 2, 3]), torch.tensor([1, 0, 3])))
```

```
tensor([False,  True, False])
```

torch.gt(a, b)

Element-wise >.

```
import torch
print(torch.gt(torch.tensor([1, 2, 3]), torch.tensor([2, 2, 2])))
```

```
tensor([False, False,  True])
```

torch.lt(a, b)

Element-wise <.

```
import torch
print(torch.lt(torch.tensor([1, 2, 3]), torch.tensor([2, 2, 2])))
```

```
tensor([ True, False, False])
```

torch.ge(a, b)

Element-wise >=.

```
import torch
print(torch.ge(torch.tensor([1, 2, 3]), torch.tensor([2, 2, 2])))
```

```
tensor([False,  True,  True])
```

torch.le(a, b)

Element-wise <=.

```
import torch
print(torch.le(torch.tensor([1, 2, 3]), torch.tensor([2, 2, 2])))
```

```
tensor([ True,  True, False])
```

torch.equal(a, b)

True if ALL elements match. Single bool, not tensor.

```
import torch
a = torch.tensor([1, 2, 3])
print(f'equal: {torch.equal(a, a.clone())}')
```

```
equal: True
```

torch.allclose(a, b)

Fuzzy equality for floats. Essential for testing!

```
import torch
a = torch.tensor([1.0])
b = torch.tensor([1.0001])
print(f'allclose: {torch.allclose(a, b, atol=1e-3)}')
```

```
allclose: True
```

torch.isnan(x)

Detect NaN values. Debug your training!

```
import torch
print(torch.isnan(torch.tensor([1., float('nan'), 3.])))
```

```
tensor([False,  True, False])
```

torch.isinf(x)

Detect infinity values.

```
import torch
print(torch.isinf(torch.tensor([1., float('inf'), float('-inf')])))
```

```
tensor([False,  True,  True])
```

torch.isfinite(x)

True for normal numbers (not inf, not nan).

```
import torch
print(torch.isfinite(torch.tensor([1., float('inf'), float('nan')])))
```

```
tensor([ True, False, False])
```

torch.isreal(x)

True if imaginary part is zero.

```
import torch
print(torch.isreal(torch.tensor([1.+0j, 1.+1j])))
```

```
tensor([ True, False])
```

torch.isclose(a, b)

Element-wise fuzzy equality. Like allclose but per-element.

```
import torch
a = torch.tensor([1.0, 2.0])
b = torch.tensor([1.001, 2.0])
print(torch.isclose(a, b, atol=1e-2))
```

```
tensor([True, True])
```

torch.maximum(a, b)

Element-wise max of two tensors.

```
import torch
print(torch.maximum(torch.tensor([1, 4]), torch.tensor([3, 2])))
```

```
tensor([3, 4])
```

torch.minimum(a, b)

Element-wise min of two tensors.

```
import torch
print(torch.minimum(torch.tensor([1, 4]), torch.tensor([3, 2])))
```

```
tensor([1, 2])
```

torch.topk(x, k)

K largest values + indices. Faster than full sort!

```
import torch
vals, idxs = torch.topk(torch.tensor([3, 1, 4, 1, 5, 9]), k=3)
```

```
print(f'top 3: values={vals}, indices={idxs}')
```

```
top 3: values=tensor([9, 5, 4]), indices=tensor([5, 4, 2])
```

torch.sort(x, dim)

Sort values + track original positions.

```
import torch
vals, idxs = torch.sort(torch.tensor([3, 1, 4, 1, 5]))
print(f'sorted: {vals}, original indices: {idxs}')
```

```
sorted: tensor([1, 1, 3, 4, 5]), original indices: tensor([1, 3, 0, 2, 4])
```

torch.argsort(x)

Indices that would sort the tensor.

```
import torch
print(torch.argsort(torch.tensor([3, 1, 4, 1, 5])))
```

```
tensor([1, 3, 0, 2, 4])
```

torch.kthvalue(x, k)

K-th smallest value.

```
import torch
val, idx = torch.kthvalue(torch.tensor([3., 1., 4., 1., 5.]), k=2)
print(f'2nd smallest: {val} at index {idx}')
```

```
2nd smallest: 1.0 at index 3
```

torch.msort(x)

Sort along first dimension. Shortcut for sort(dim=0).

```
import torch
print(torch.msort(torch.tensor([3, 1, 4, 1, 5])))
```

```
tensor([1, 1, 3, 4, 5])
```

torch.searchsorted(sorted, values)

Find insertion points in sorted sequence. Like numpy.

```
import torch
sorted_seq = torch.tensor([1., 3., 5., 7.])
print(torch.searchsorted(sorted_seq, torch.tensor([2., 4.])))
```

```
tensor([1, 2])
```

torch.unique(x)

Unique elements, sorted.

```
import torch
x = torch.tensor([1, 2, 2, 3, 1, 3])
print(f'unique: {torch.unique(x)}')
```

```
unique: tensor([1, 2, 3])
```

torch.unique_consecutive(x)

Remove consecutive duplicates. Faster than unique.

```
import torch
print(torch.unique_consecutive(torch.tensor([1, 1, 2, 2, 1])))
```

```
tensor([1, 2, 1])
```

!Logic Chart

10. Math -- Logic

10 functions with examples | 10 total in this category

torch.logical_and(a, b)

Element-wise AND.

```
import torch
a = torch.tensor([True, True, False])
b = torch.tensor([True, False, False])
print(torch.logical_and(a, b))
```

```
tensor([ True, False, False])
```

torch.logical_or(a, b)

Element-wise OR.

```
import torch
print(torch.logical_or(torch.tensor([True, False]), torch.tensor([False, False])))
```

```
tensor([ True, False])
```

torch.logical_not(x)

Element-wise NOT.

```
import torch
print(torch.logical_not(torch.tensor([True, False, True])))
```

```
tensor([False,  True, False])
```

torch.logical_xor(a, b)

Element-wise XOR.

```
import torch
print(torch.logical_xor(torch.tensor([True, True]), torch.tensor([True, False])))
```

```
tensor([False,  True])
```

torch.bitwise_and(a, b)

Bitwise AND on integers.

```
import torch
print(torch.bitwise_and(torch.tensor([5]), torch.tensor([3])))  # 101 & 011 = 001
```

```
tensor([1])
```

torch.bitwise_or(a, b)

Bitwise OR.

```
import torch
print(torch.bitwise_or(torch.tensor([5]), torch.tensor([3])))  # 101 | 011 = 111
```

```
tensor([7])
```

torch.bitwise_not(x)

Bitwise NOT (complement).

```
import torch
print(torch.bitwise_not(torch.tensor([0, 1, -1], dtype=torch.int8)))
```

```
tensor([-1, -2,  0], dtype=torch.int8)
```

torch.bitwise_xor(a, b)

Bitwise XOR.

```
import torch
print(torch.bitwise_xor(torch.tensor([5]), torch.tensor([3])))
```

```
tensor([6])
```

torch.bitwise_left_shift(a, b)

Shift bits left. Equivalent to 2^n.*

```
import torch
print(torch.bitwise_left_shift(torch.tensor([1, 2]), torch.tensor([1, 2])))
```

```
tensor([2, 8])
```

torch.bitwise_right_shift(a, b)

Shift bits right. Equivalent to // 2^n.

```
import torch
print(torch.bitwise_right_shift(torch.tensor([8, 16]), torch.tensor([1, 2])))
```

```
tensor([4, 4])
```

Part III: Core

!Linear Algebra Chart

11. Linear Algebra

26 functions with examples | 26 total in this category

torch.mm(a, b)

2D matrix multiply. Use matmul for higher dims.

```
import torch
a = torch.tensor([[1., 2.], [3., 4.]])
b = torch.tensor([[5., 6.], [7., 8.]])
print(torch.mm(a, b))
```

```
tensor([[19., 22.],
        [43., 50.]])
```

torch.bmm(a, b)

Batch matrix multiply. First dim is batch.

```
import torch
a = torch.randn(2, 3, 4)
b = torch.randn(2, 4, 5)
print(f'bmm: {torch.bmm(a, b).shape}')
```

```
bmm: torch.Size([2, 3, 5])
```

torch.matmul(a, b)

Swiss army knife: dot product, matrix mult, batch mult. Use @ operator!

```
import torch
print(f'1D dot: {torch.matmul(torch.tensor([1.,2.]), torch.tensor([3.,4.]))}')
print(f'2D mm:  {torch.matmul(torch.randn(2,3), torch.randn(3,4)).shape}')
```

```
1D dot: 11.0
2D mm:  torch.Size([2, 4])
```

torch.mv(mat, vec)

Matrix-vector multiplication.

```
import torch
M = torch.tensor([[1., 2.], [3., 4.]])
v = torch.tensor([1., 1.])
print(torch.mv(M, v))
```

```
tensor([3., 7.])
```

torch.dot(a, b)

Dot product of 1D tensors. Sum of element-wise products.

```
import torch
print(torch.dot(torch.tensor([1., 2., 3.]), torch.tensor([4., 5., 6.])))
```

```
tensor(32.)
```

torch.outer(a, b)

Outer product: every pair of elements multiplied.

```
import torch
print(torch.outer(torch.tensor([1., 2., 3.]), torch.tensor([4., 5.])))
```

```
tensor([[ 4.,  5.],
        [ 8., 10.],
        [12., 15.]])
```

torch.inner(a, b)

Inner product (same as dot for 1D).

```
import torch
print(torch.inner(torch.tensor([1., 2.]), torch.tensor([3., 4.])))
```

```
tensor(11.)
```

torch.cross(a, b, dim)

Cross product of 3D vectors. i x j = k!

```
import torch
print(torch.cross(torch.tensor([1., 0., 0.]), torch.tensor([0., 1., 0.])))
```

```
tensor([0., 0., 1.])
```

torch.einsum(eq, *tensors)

Einstein summation. The most powerful operation in PyTorch!

```
import torch
a = torch.randn(2, 3)
b = torch.randn(3, 4)
print(f'matmul via einsum: {torch.einsum("ij,jk->ik", a, b).shape}')
print(f'trace: {torch.einsum("ii->", torch.eye(3))}')
```

```
matmul via einsum: torch.Size([2, 4])
trace: 3.0
```

torch.det(x) / torch.linalg.det(x)

Matrix determinant.

```
import torch
print(f'det: {torch.det(torch.tensor([[1., 2.], [3., 4.]])):.1f}')
```

```
det: -2.0
```

torch.inverse(x) / torch.linalg.inv(x)

Matrix inverse. A @ A^-1 = I.

```
import torch
A = torch.tensor([[1., 2.], [3., 4.]])
inv = torch.linalg.inv(A)
print(f'A @ inv =\n{(A @ inv).round()}')
```

```
A @ inv =
tensor([[1., 0.],
        [0., 1.]])
```

torch.linalg.svd(x)

Singular Value Decomposition. The backbone of PCA!

```
import torch
```

```
U, S, Vh = torch.linalg.svd(torch.randn(3, 2))
print(f'U:{U.shape} S:{S} Vh:{Vh.shape}')
```

```
U:torch.Size([3, 3]) S:tensor([1.5227, 0.4568]) Vh:torch.Size([2, 2])
```

torch.linalg.eigh(x)

Eigenvalues of symmetric matrix. For general: use torch.linalg.eig.

```
import torch
vals, vecs = torch.linalg.eigh(torch.tensor([[2., 1.], [1., 2.]]))
print(f'eigenvalues: {vals}')
```

```
eigenvalues: tensor([1., 3.])
```

torch.linalg.eigvalsh(x)

Eigenvalues only (no eigenvectors). Faster.

```
import torch
print(torch.linalg.eigvalsh(torch.tensor([[2., 1.], [1., 2.]])))
```

```
tensor([1., 3.])
```

torch.linalg.qr(x)

QR decomposition. Q orthogonal, R upper triangular.

```
import torch
Q, R = torch.linalg.qr(torch.randn(3, 2))
print(f'Q:{Q.shape} R:{R.shape}')
```

```
Q:torch.Size([3, 2]) R:torch.Size([2, 2])
```

torch.linalg.cholesky(x)

Cholesky decomposition of positive-definite matrix. A = LL^T.

```
import torch
A = torch.tensor([[4., 2.], [2., 3.]])
L = torch.linalg.cholesky(A)
print(f'L @ L.T =\n{L @ L.T}')
```

```
L @ L.T =
tensor([[4., 2.],
        [2., 3.]])
```

torch.linalg.solve(A, b)

Solve Ax = b. Better than computing inverse!

```
import torch
A = torch.tensor([[3., 1.], [1., 2.]])
b = torch.tensor([9., 8.])
print(f'x = {torch.linalg.solve(A, b)}')
```

```
x = tensor([2., 3.])
```

torch.linalg.lstsq(A, b)

Least squares solution. For overdetermined systems.

```
import torch
A = torch.tensor([[1., 1.], [1., 2.], [1., 3.]])
b = torch.tensor([1., 2., 2.])
result = torch.linalg.lstsq(A, b)
print(f'Least squares solution: {result.solution}')
```

```
Least squares solution: tensor([0.6667, 0.5000])
```

torch.linalg.norm(x, ord)

Matrix/vector norms. Default is Frobenius for matrices.

```
import torch
x = torch.tensor([[1., 2.], [3., 4.]])
print(f'Frobenius: {torch.linalg.norm(x):.4f}')
```

```
Frobenius: 5.4772
```

torch.linalg.matrix_norm(x)

Specifically for matrix norms (Frobenius, nuclear, etc.).

```
import torch
A = torch.randn(3, 3)
print(f'matrix norm: {torch.linalg.matrix_norm(A):.4f}')
```

```
matrix norm: 2.9604
```

torch.linalg.matrix_rank(x)

Number of linearly independent rows/columns.

```
import torch
A = torch.tensor([[1., 2.], [2., 4.]])  # rank 1
print(f'rank: {torch.linalg.matrix_rank(A)}')
```

```
rank: 1
```

torch.linalg.pinv(x)

Moore-Penrose pseudo-inverse. Works for non-square matrices.

```
import torch
A = torch.randn(3, 2)
print(f'pseudo-inverse: {torch.linalg.pinv(A).shape}')
```

```
pseudo-inverse: torch.Size([2, 3])
```

torch.trace(x)

Sum of diagonal elements.

```
import torch
print(f'trace: {torch.trace(torch.tensor([[1., 2.], [3., 4.]]))}')
```

```
trace: 5.0
```

torch.diag(x, diagonal)

1D->diagonal matrix, 2D->extract diagonal.

```
import torch
print(f'diag matrix:\n{torch.diag(torch.tensor([1, 2, 3]))}')
```

```
diag matrix:
tensor([[1, 0, 0],
        [0, 2, 0],
        [0, 0, 3]])
```

torch.linalg.matrix_exp(x)

Matrix exponential. e^A (not element-wise!).

```
import torch
A = torch.tensor([[0., 1.], [-1., 0.]])  # rotation
print(f'matrix exp:\n{torch.linalg.matrix_exp(A)}')
```

```
matrix exp:
tensor([[ 0.5403,  0.8415],
        [-0.8415,  0.5403]])
```

torch.linalg.multi_dot(tensors)

Chain of matrix multiplications. Optimizes order automatically!

```
import torch
A = torch.randn(2, 3)
B = torch.randn(3, 4)
C = torch.randn(4, 2)
print(f'multi_dot: {torch.linalg.multi_dot([A, B, C]).shape}')
```

```
multi_dot: torch.Size([2, 2])
```

!Random Sampling Chart

12. Random Sampling

14 functions with examples | 15 total in this category

torch.manual_seed(seed)

Set seed for reproducibility. Same seed = same numbers.

```
import torch
torch.manual_seed(42)
print(torch.rand(3))
torch.manual_seed(42)
print(torch.rand(3))  # same!
```

```
tensor([0.8823, 0.9150, 0.3829])
tensor([0.8823, 0.9150, 0.3829])
```

torch.Generator()

Independent RNG state. Useful for data loading.

```
import torch
g = torch.Generator().manual_seed(42)
print(torch.rand(3, generator=g))
```

```
tensor([0.8823, 0.9150, 0.3829])
```

torch.rand(*size)

Uniform random in [0, 1). The workhorse of randomness.

```
import torch
torch.manual_seed(42)
print(torch.rand(2, 3))
```

```
tensor([[0.8823, 0.9150, 0.3829],
        [0.9593, 0.3904, 0.6009]])
```

torch.randn(*size)

Normal distribution N(0, 1). Used for weight initialization.

```
import torch
torch.manual_seed(42)
print(torch.randn(2, 3))
```

```
tensor([[ 0.3367,  0.1288,  0.2345],
        [ 0.2303, -1.1229, -0.1863]])
```

• torch.randint(lo, hi, size)

torch.randperm(n)

Random permutation of 0..n-1. Perfect for shuffling.

```
import torch
torch.manual_seed(42)
print(torch.randperm(8))
```

```
tensor([6, 3, 0, 7, 2, 1, 4, 5])
```

torch.bernoulli(p)

Coin flip: 1 with probability p, 0 otherwise.

```
import torch
torch.manual_seed(42)
print(torch.bernoulli(torch.full((10,), 0.3)))
```

```
tensor([0., 0., 0., 0., 0., 0., 1., 0., 0., 1.])
```

torch.multinomial(weights, n)

Sample from weighted distribution. Core of text generation!

```
import torch
weights = torch.tensor([0.1, 0.3, 0.6])
torch.manual_seed(42)
print(torch.multinomial(weights, 5, replacement=True))
```

```
tensor([0, 0, 1, 0, 2])
```

torch.poisson(rates)

Poisson-distributed random numbers.

```
import torch
torch.manual_seed(42)
print(torch.poisson(torch.tensor([1., 5., 10.])))
```

```
tensor([ 0.,  2., 10.])
```

torch.normal(mean, std)

Normal distribution with specified mean and std.

```
import torch
torch.manual_seed(42)
print(torch.normal(mean=0., std=1., size=(5,)))
```

```
tensor([ 0.3367,  0.1288,  0.2345,  0.2303, -1.1229])
```

x.uniform_(lo, hi)

Fill tensor with uniform random values in-place.

```
import torch
torch.manual_seed(42)
x = torch.empty(5).uniform_(-1, 1)
print(x)
```

```
tensor([ 0.7645,  0.8300, -0.2343,  0.9186, -0.2191])
```

x.normal_(mean, std)

Fill with normal random values in-place.

```
import torch
torch.manual_seed(42)
x = torch.empty(5).normal_(0, 1)
print(x)
```

```
tensor([ 0.3367,  0.1288,  0.2345,  0.2303, -1.1229])
```

x.random_(lo, hi)

Fill with random integers in-place.

```
import torch
torch.manual_seed(42)
x = torch.empty(5, dtype=torch.long).random_(0, 10)
print(x)
```

```
tensor([2, 7, 6, 4, 6])
```

x.bernoulli_(p)

In-place Bernoulli sampling.

```
import torch
torch.manual_seed(42)
x = torch.empty(10).bernoulli_(0.5)
print(x)
```

```
tensor([1., 1., 1., 1., 0., 1., 0., 0., 1., 1.])
```

torch.initial_seed()

Get the current random seed.

```
import torch
```

```
torch.manual_seed(42)
print(f'current seed: {torch.initial_seed()}')
```

```
current seed: 42
```

!Autograd Chart

13. Autograd

15 functions with examples | 15 total in this category

x.requires_grad_(True)

Enable gradient tracking. The on-switch for backprop.

```
import torch
x = torch.tensor([1., 2., 3.])
x.requires_grad_(True)
print(f'tracking gradients: {x.requires_grad}')
```

```
tracking gradients: True
```

y.backward()

Compute gradients via backpropagation.

```
import torch
x = torch.tensor([2., 3.], requires_grad=True)
y = (x ** 2).sum()
y.backward()
print(f'dy/dx = 2x = {x.grad}')
```

```
dy/dx = 2x = tensor([4., 6.])
```

x.grad

Gradient stored after backward(). Accumulates!

```
import torch
x = torch.tensor(3., requires_grad=True)
(x ** 3).backward()
print(f'x={x}, d(x^3)/dx = 3x^2 = {x.grad}')
```

```
x=3.0, d(x^3)/dx = 3x^2 = 27.0
```

x.detach()

Cut tensor from computation graph. No gradients flow back.

```
import torch
x = torch.tensor([1.], requires_grad=True)
y = (x * 2).detach()
print(f'detached: requires_grad={y.requires_grad}')
```

```
detached: requires_grad=False
```

x.clone()

Deep copy. Changes to clone don't affect original.

```
import torch
x = torch.tensor([1., 2.])
y = x.clone()
y[0] = 99
print(f'x={x}, y={y}  # independent copies')
```

```
x=tensor([1., 2.]), y=tensor([99.,  2.])  # independent copies
```

torch.no_grad()

Disable grad computation. Use for inference = faster + less memory.

```
import torch
x = torch.tensor([1.], requires_grad=True)
with torch.no_grad():
    print(f'in no_grad: {(x*2).requires_grad}')
print(f'outside:    {(x*2).requires_grad}')
```

```
in no_grad: False
outside:    True
```

torch.enable_grad()

Re-enable gradients inside a no_grad block.

```
import torch
with torch.no_grad():
    with torch.enable_grad():
        x = torch.tensor([1.], requires_grad=True)
        print(f'enabled inside no_grad: {(x*2).requires_grad}')
```

```
enabled inside no_grad: True
```

torch.set_grad_enabled(mode)

Programmatically toggle gradient computation.

```
import torch
torch.set_grad_enabled(False)
x = torch.tensor([1.], requires_grad=True)
print(f'disabled: {(x*2).requires_grad}')
torch.set_grad_enabled(True)
```

```
disabled: False
```

torch.is_grad_enabled()

Check if gradient computation is enabled.

```
import torch
print(f'grad enabled: {torch.is_grad_enabled()}')
```

```
grad enabled: True
```

torch.autograd.grad(y, x)

Compute gradient without .grad attribute. Functional style.

```
import torch
x = torch.tensor([1., 2.], requires_grad=True)
y = (x ** 3).sum()
grad = torch.autograd.grad(y, x)
print(f'3x^2 = {grad[0]}')
```

```
3x^2 = tensor([ 3., 12.])
```

torch.autograd.backward(tensors)

Explicit backward with gradient tensor.

```
import torch
x = torch.tensor([1., 2.], requires_grad=True)
y = x ** 2
torch.autograd.backward(y, torch.ones_like(y))
print(f'grad: {x.grad}')
```

```
grad: tensor([2., 4.])
```

torch.autograd.functional.jacobian(f, x)

Full Jacobian matrix. Expensive but powerful.

```
import torch
from torch.autograd.functional import jacobian
def f(x): return torch.stack([x[0]**2, x[0]*x[1]])
J = jacobian(f, torch.tensor([1., 2.]))
print(f'Jacobian:\n{J}')
```

```
Jacobian:
tensor([[2., 0.],
        [2., 1.]])
```

torch.autograd.functional.hessian(f, x)

Second-order derivatives. The curvature of the loss surface.

```
import torch
from torch.autograd.functional import hessian
def f(x): return (x**3).sum()
H = hessian(f, torch.tensor([1., 2.]))
print(f'Hessian:\n{H}')
```

```
Hessian:
tensor([[ 6.,  0.],
        [ 0., 12.]])
```

torch.autograd.functional.vjp(f, x, v)

Vector-Jacobian product. What backward() computes!

```
import torch
from torch.autograd.functional import vjp
def f(x): return x ** 2
out, vjp_val = vjp(f, torch.tensor([1., 2.]), torch.ones(2))
print(f'vjp: {vjp_val}')
```

```
vjp: tensor([2., 4.])
```

torch.autograd.functional.jvp(f, x, v)

Jacobian-vector product. Forward-mode autodiff.

```
import torch
from torch.autograd.functional import jvp
def f(x): return x ** 2
out, jvp_val = jvp(f, (torch.tensor([1., 2.]),), (torch.ones(2),))
print(f'jvp: {jvp_val}')
```

```
jvp: tensor([2., 4.])
```

!Device Memory Chart

14. Device & Memory

16 functions with examples | 16 total in this category

torch.cuda.is_available()

Check GPU availability. First thing in any training script!

```
import torch
print(f'CUDA: {torch.cuda.is_available()}')
print(f'MPS:  {torch.backends.mps.is_available()}')
```

```
CUDA: False
MPS:  True
```

torch.backends.mps.is_available()

Apple Metal Performance Shaders for M1/M2/M3.

```
import torch
if torch.backends.mps.is_available():
    print('Apple Silicon GPU available!')
else:
    print('MPS not available')
```

```
Apple Silicon GPU available!
```

torch.device(name)

The standard device selection pattern.

```
import torch
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using: {device}')
```

```
Using: cpu
```

x.to(device)

Move tensor to device. Also works with dtype!

```
import torch
x = torch.tensor([1., 2.])
print(f'{x.device} -> {x.to("cpu").device}')
```

```
cpu -> cpu
```

x.cpu()

Shortcut to move to CPU.

```
import torch
print(torch.tensor([1.]).cpu().device)
```

```
cpu
```

x.cuda()

Shortcut to move to CUDA GPU.

```
import torch
if torch.cuda.is_available():
    print(torch.tensor([1.]).cuda().device)
```

```
else:
    print('No CUDA (would move to GPU)')
```

```
No CUDA (would move to GPU)
```

torch.cuda.device_count()

How many GPUs you have.

```
import torch
print(f'GPUs: {torch.cuda.device_count() if torch.cuda.is_available() else 0}')
```

```
GPUs: 0
```

torch.cuda.current_device()

Which GPU is currently active.

```
import torch
if torch.cuda.is_available():
    print(f'Current device: {torch.cuda.current_device()}')
else:
    print('No CUDA device')
```

```
No CUDA device
```

torch.cuda.get_device_name()

Human-readable GPU name (e.g., 'NVIDIA A100').

```
import torch
if torch.cuda.is_available():
    print(torch.cuda.get_device_name(0))
else:
    print('No GPU to name')
```

```
No GPU to name
```

torch.cuda.empty_cache()

Release unused GPU memory. Helps with OOM!

```
import torch
if torch.cuda.is_available():
    torch.cuda.empty_cache()
    print('Cache cleared')
else:
    print('No CUDA cache to clear')
```

```
No CUDA cache to clear
```

torch.cuda.memory_allocated()

Current GPU memory used by tensors.

```
import torch
if torch.cuda.is_available():
    print(f'{torch.cuda.memory_allocated() / 1e6:.1f} MB')
else:
    print('No CUDA memory tracking')
```

```
No CUDA memory tracking
```

torch.cuda.max_memory_allocated()

Peak GPU memory usage. Find your memory bottleneck!

```
import torch
if torch.cuda.is_available():
    print(f'Peak: {torch.cuda.max_memory_allocated() / 1e6:.1f} MB')
else:
    print('No CUDA (tracks peak memory)')
```

```
No CUDA (tracks peak memory)
```

torch.cuda.memory_summary()

Full memory report. Amazing for debugging OOM.

```
import torch
if torch.cuda.is_available():
    print(torch.cuda.memory_summary(abbreviated=True))
else:
    print('Detailed GPU memory report (CUDA only)')
```

```
Detailed GPU memory report (CUDA only)
```

torch.cuda.synchronize()

Wait for GPU. Needed for accurate timing!

```
import torch
if torch.cuda.is_available():
    torch.cuda.synchronize()
    print('GPU ops complete')
else:
    print('Waits for all GPU ops to finish')
```

```
Waits for all GPU ops to finish
```

x.pin_memory()

Page-locked memory for faster CPU->GPU transfer.

```
import torch
x = torch.randn(1000)
y = x.pin_memory()
```

```
print(f'pinned: {y.is_pinned()}')
```

```
Error: Attempted to set the storage of a tensor on device "cpu" to a storage on different device "mps:0".  This is r
```

x.is_pinned()

Check if tensor is in pinned memory.

```
import torch
print(f'pinned: {torch.tensor([1.]).is_pinned()}')
```

```
pinned: False
```

!Serialization Chart

15. Serialization

6 functions with examples | 6 total in this category

torch.save(obj, path)

Save anything: tensors, models, dicts, optimizers.

```
import torch, os
torch.save(torch.randn(100, 100), '/tmp/t.pt')
print(f'Saved: {os.path.getsize("/tmp/t.pt"):,} bytes')
```

```
Saved: 41,343 bytes
```

torch.load(path)

Load saved objects. Use weights_only=True for untrusted files!

```
import torch
torch.save({'a': 1, 'b': torch.tensor([1,2])}, '/tmp/d.pt')
print(torch.load('/tmp/d.pt', weights_only=False))
```

```
{'a': 1, 'b': tensor([1, 2])}
```

model.state_dict()

Dict of all learnable parameters. The recommended way to save models.

```
import torch.nn as nn
m = nn.Linear(5, 3)
for k, v in m.state_dict().items():
    print(f'{k}: {v.shape}')
```

```
weight: torch.Size([3, 5])
bias: torch.Size([3])
```

model.load_state_dict(sd)

Load weights into a model. Architecture must match!

```
import torch, torch.nn as nn
m1 = nn.Linear(5, 3)
torch.save(m1.state_dict(), '/tmp/m.pt')
m2 = nn.Linear(5, 3)
m2.load_state_dict(torch.load('/tmp/m.pt', weights_only=True))
print(f'Match: {torch.equal(m1.weight, m2.weight)}')
```

```
Match: True
```

torch.jit.save(m, path)

Save JIT-compiled model. No Python needed to load!

```
import torch
m = torch.jit.script(torch.nn.Linear(5, 3))
torch.jit.save(m, '/tmp/jit.pt')
print('TorchScript model saved')
```

```
TorchScript model saved
```

torch.jit.load(path)

Load JIT model. Works in C++ too!

```
import torch
m = torch.jit.script(torch.nn.Linear(5, 3))
torch.jit.save(m, '/tmp/jit.pt')
m2 = torch.jit.load('/tmp/jit.pt')
print(f'Loaded: {m2}')
```

```
Loaded: RecursiveScriptModule(original_name=Linear)
```

Part IV: Neural Networks

!Module Layers Chart

16. nn.Module & Layers

21 functions with examples | 21 total in this category

nn.Module

Base class for ALL neural networks. Override __init__ + forward.

```
import torch, torch.nn as nn
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(10, 1)
    def forward(self, x):
        return self.fc(x)
model = Net()
print(f'Params: {sum(p.numel() for p in model.parameters())}')
```

```
Params: 11
```

nn.Sequential

Stack layers in order. Clean and simple.

```
import torch, torch.nn as nn
model = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 1))
print(model)
print(f'Output: {model(torch.randn(2, 10)).shape}')
```

```
Sequential(
  (0): Linear(in_features=10, out_features=32, bias=True)
  (1): ReLU()
  (2): Linear(in_features=32, out_features=1, bias=True)
)
Output: torch.Size([2, 1])
```

nn.ModuleList

List of modules. Registers params properly (plain list doesn't!).

```
import torch.nn as nn
layers = nn.ModuleList([nn.Linear(10, 10) for _ in range(3)])
print(f'{len(layers)} layers, {sum(p.numel() for p in layers.parameters())} params')
```

```
3 layers, 330 params
```

nn.ModuleDict

Dict of modules with proper parameter registration.

```
import torch.nn as nn
blocks = nn.ModuleDict({'enc': nn.Linear(10, 5), 'dec': nn.Linear(5, 10)})
print(blocks['enc'])
```

```
Linear(in_features=10, out_features=5, bias=True)
```

nn.Linear(in, out)

Fully connected: y = xW^T + b. The most basic layer.

```
import torch, torch.nn as nn
l = nn.Linear(5, 3)
print(f'Weight: {l.weight.shape}, Bias: {l.bias.shape}')
print(f'Output: {l(torch.randn(2, 5)).shape}')
```

```
Weight: torch.Size([3, 5]), Bias: torch.Size([3])
Output: torch.Size([2, 3])
```

nn.Bilinear(in1, in2, out)

Bilinear transform: y = x1^T A x2 + b.

```
import torch, torch.nn as nn
bl = nn.Bilinear(5, 4, 3)
print(f'Output: {bl(torch.randn(2, 5), torch.randn(2, 4)).shape}')
```

```
Output: torch.Size([2, 3])
```

nn.Embedding(num, dim)

Lookup table: integer -> dense vector. The first layer of every NLP model!

```
import torch, torch.nn as nn
emb = nn.Embedding(1000, 64)  # vocab=1000, dim=64
idx = torch.tensor([0, 42, 999])
print(f'Embedding: {emb(idx).shape}')
```

```
Embedding: torch.Size([3, 64])
```

nn.EmbeddingBag(num, dim)

Embedding + reduction in one step. Efficient for bag-of-words.

```
import torch, torch.nn as nn
ebag = nn.EmbeddingBag(100, 16, mode='mean')
print(f'Output: {ebag(torch.tensor([0, 1, 2, 3]), torch.tensor([0, 2])).shape}')
```

```
Output: torch.Size([2, 16])
```

nn.Parameter

Wrap tensor as learnable parameter. Autograd + optimizer aware.

```
import torch, torch.nn as nn
class Scale(nn.Module):
    def __init__(self):
        super().__init__()
        self.s = nn.Parameter(torch.ones(3))
    def forward(self, x): return x * self.s
m = Scale()
print(list(m.parameters()))
```

```
[Parameter containing:
tensor([1., 1., 1.], requires_grad=True)]
```

nn.LazyLinear(out)

Linear that infers input size on first forward. Lazy = smart!

```
import torch, torch.nn as nn
l = nn.LazyLinear(5)
print(f'Before: weight not materialized')
out = l(torch.randn(2, 10))
print(f'After: weight={l.weight.shape}')
```

```
Before: weight not materialized
After: weight=torch.Size([5, 10])
```

model.parameters()

Iterator over all parameters. Feed this to your optimizer.

```
import torch.nn as nn
m = nn.Sequential(nn.Linear(10, 5), nn.Linear(5, 1))
for p in m.parameters():
    print(p.shape)
```

```
torch.Size([5, 10])
torch.Size([5])
torch.Size([1, 5])
torch.Size([1])
```

model.named_parameters()

Parameters with their names. Great for freezing specific layers.

```
import torch.nn as nn
m = nn.Sequential(nn.Linear(10, 5), nn.Linear(5, 1))
for name, p in m.named_parameters():
    print(f'{name}: {p.shape}')
```

```
0.weight: torch.Size([5, 10])
0.bias: torch.Size([5])
1.weight: torch.Size([1, 5])
1.bias: torch.Size([1])
```

model.children()

Direct child modules (not recursive).

```
import torch.nn as nn
m = nn.Sequential(nn.Linear(10, 5), nn.ReLU(), nn.Linear(5, 1))
for child in m.children():
    print(child)
```

```
Linear(in_features=10, out_features=5, bias=True)
```

```
ReLU()
Linear(in_features=5, out_features=1, bias=True)
```

model.named_children()

Named direct children.

```
import torch.nn as nn
m = nn.Sequential(nn.Linear(10, 5), nn.ReLU())
for name, child in m.named_children():
    print(f'{name}: {child}')
```

```
0: Linear(in_features=10, out_features=5, bias=True)
1: ReLU()
```

model.modules()

ALL modules recursively (including self).

```
import torch.nn as nn
m = nn.Sequential(nn.Linear(10, 5), nn.ReLU())
print(f'Total modules: {len(list(m.modules()))}')
```

```
Total modules: 3
```

model.named_modules()

All modules with dot-separated names.

```
import torch.nn as nn
m = nn.Sequential(nn.Linear(10, 5), nn.ReLU())
for name, mod in m.named_modules():
    print(f'{name or "root"}: {mod.__class__.__name__}')
```

```
root: Sequential
0: Linear
1: ReLU
```

model.train()

Enable training mode (dropout active, batchnorm uses batch stats).

```
import torch.nn as nn
m = nn.Sequential(nn.Linear(10, 5), nn.Dropout(0.5))
m.train()
print(f'Training mode: {m.training}')
```

```
Training mode: True
```

model.eval()

Enable eval mode (dropout disabled, batchnorm uses running stats).

```
import torch.nn as nn
m = nn.Sequential(nn.Linear(10, 5), nn.Dropout(0.5))
m.eval()
print(f'Eval mode: {not m.training}')
```

```
Eval mode: True
```

model.apply(fn)

Apply function to every module. Perfect for weight initialization!

```
import torch, torch.nn as nn
m = nn.Sequential(nn.Linear(10, 5), nn.Linear(5, 1))
def init(layer):
    if isinstance(layer, nn.Linear): nn.init.zeros_(layer.weight)
m.apply(init)
print(f'All zeros: {m[0].weight.sum()}')
```

```
All zeros: 0.0
```

model.zero_grad()

Zero all parameter gradients. Usually optimizer.zero_grad() instead.

```
import torch, torch.nn as nn
m = nn.Linear(5, 1)
m(torch.randn(1, 5)).backward()
print(f'Before: {m.weight.grad.sum():.4f}')
m.zero_grad()
print(f'After:  {m.weight.grad.sum()}')
```

```
Error: 'NoneType' object has no attribute 'sum'
```

model.register_buffer(name, tensor)

Non-learnable tensor saved with model. Moves with .to(device).

```
import torch, torch.nn as nn
class M(nn.Module):
    def __init__(self):
        super().__init__()
        self.register_buffer('mask', torch.ones(3))
m = M()
print(f'buffer in state_dict: {"mask" in m.state_dict()}')
```

```
buffer in state_dict: True
```

!Activation Functions Chart

17. Activation Functions

20 functions with examples | 20 total in this category

nn.ReLU()

max(0, x). The default activation. Simple and effective.

```
import torch, torch.nn as nn
x = torch.tensor([-2., -1., 0., 1., 2.])
print(f'ReLU: {nn.ReLU()(x)}')
```

```
ReLU: tensor([0., 0., 0., 1., 2.])
```

nn.ReLU6()

min(max(0, x), 6). Used in MobileNet.

```
import torch, torch.nn as nn
print(nn.ReLU6()(torch.tensor([-1., 3., 7.])))
```

```
tensor([0., 3., 6.])
```

nn.LeakyReLU(slope)

Small slope for negatives. Prevents dying neurons.

```
import torch, torch.nn as nn
print(nn.LeakyReLU(0.1)(torch.tensor([-2., -1., 0., 1.])))
```

```
tensor([-0.2000, -0.1000,  0.0000,  1.0000])
```

nn.PReLU()

Parametric ReLU: slope is LEARNED, not fixed.

```
import torch, torch.nn as nn
prelu = nn.PReLU()
print(f'Learnable slope: {prelu.weight.item()}')
```

```
Learnable slope: 0.25
```

nn.ELU(alpha)

Exponential linear unit. Smooth below zero.

```
import torch, torch.nn as nn
print(nn.ELU()(torch.tensor([-2., -1., 0., 1.])))
```

```
tensor([-0.8647, -0.6321,  0.0000,  1.0000])
```

nn.SELU()

Self-normalizing. Automatically maintains mean=0, var=1.

```
import torch, torch.nn as nn
print(nn.SELU()(torch.tensor([-2., 0., 2.])))
```

```
tensor([-1.5202,  0.0000,  2.1014])
```

nn.GELU()

Gaussian Error Linear Unit. THE activation for transformers.

```
import torch, torch.nn as nn
print(nn.GELU()(torch.tensor([-2., -1., 0., 1., 2.])))
```

```
tensor([-0.0455, -0.1587,  0.0000,  0.8413,  1.9545])
```

nn.SiLU()

SiLU/Swish: x sigmoid(x). Used in modern architectures.*

```
import torch, torch.nn as nn
print(nn.SiLU()(torch.tensor([-2., 0., 2.])))
```

```
tensor([-0.2384,  0.0000,  1.7616])
```

nn.Mish()

x tanh(softplus(x)). Smooth and non-monotonic.*

```
import torch, torch.nn as nn
print(nn.Mish()(torch.tensor([-2., 0., 2.])))
```

```
tensor([-0.2525,  0.0000,  1.9440])
```

nn.Sigmoid()

Squash to (0, 1). Binary classification output layer.

```
import torch, torch.nn as nn
print(nn.Sigmoid()(torch.tensor([-2., 0., 2.])))
```

```
tensor([0.1192, 0.5000, 0.8808])
```

nn.Tanh()

Squash to (-1, 1). Zero-centered.

```
import torch, torch.nn as nn
print(nn.Tanh()(torch.tensor([-2., 0., 2.])))
```

```
tensor([-0.9640,  0.0000,  0.9640])
```

nn.Hardswish()

Efficient approximation of SiLU. Used in MobileNetV3.

```
import torch, torch.nn as nn
print(nn.Hardswish()(torch.tensor([-3., 0., 3.])))
```

```
tensor([-0., 0., 3.])
```

nn.Hardsigmoid()

Piecewise linear approximation of sigmoid. Fast!

```
import torch, torch.nn as nn
print(nn.Hardsigmoid()(torch.tensor([-3., 0., 3.])))
```

```
tensor([0.0000, 0.5000, 1.0000])
```

nn.Softmax(dim)

Logits -> probabilities (sum = 1). Classification output.

```
import torch, torch.nn as nn
logits = torch.tensor([2., 1., 0.1])
print(f'probs: {nn.Softmax(dim=0)(logits)}, sum={nn.Softmax(dim=0)(logits).sum():.1f}')
```

```
probs: tensor([0.6590, 0.2424, 0.0986]), sum=1.0
```

nn.LogSoftmax(dim)

log(softmax(x)). More numerically stable. Pair with NLLLoss.

```
import torch, torch.nn as nn
print(nn.LogSoftmax(dim=0)(torch.tensor([2., 1., 0.1])))
```

```
tensor([-0.4170, -1.4170, -2.3170])
```

nn.Softmin(dim)

softmax(-x). Gives high probability to smallest values.

```
import torch, torch.nn as nn
print(nn.Softmin(dim=0)(torch.tensor([2., 1., 0.1])))
```

```
tensor([0.0961, 0.2613, 0.6426])
```

nn.Softplus(beta)

Smooth approximation of ReLU: log(1 + exp(x)).

```
import torch, torch.nn as nn
print(nn.Softplus()(torch.tensor([-2., 0., 2.])))
```

```
tensor([0.1269, 0.6931, 2.1269])
```

nn.Softsign()

x / (1 + |x|). Like tanh but with polynomial tails.

```
import torch, torch.nn as nn
print(nn.Softsign()(torch.tensor([-2., 0., 2.])))
```

```
tensor([-0.6667,  0.0000,  0.6667])
```

nn.Threshold(threshold, value)

Replace values below threshold. Generalized ReLU.

```
import torch, torch.nn as nn
print(nn.Threshold(0.5, 0)(torch.tensor([0.2, 0.5, 0.8])))
```

```
tensor([0.0000, 0.0000, 0.8000])
```

nn.GLU(dim)

Gated Linear Unit: splits input, applies sigmoid gate.

```
import torch, torch.nn as nn
x = torch.randn(2, 10)
print(f'{x.shape} -> GLU: {nn.GLU(dim=1)(x).shape}')
```

```
torch.Size([2, 10]) -> GLU: torch.Size([2, 5])
```

!Loss Functions Chart

18. Loss Functions

19 functions with examples | 19 total in this category

nn.MSELoss()

Mean Squared Error. THE regression loss.

```
import torch, torch.nn as nn
pred = torch.tensor([1., 2., 3.])
tgt = torch.tensor([1.5, 2., 2.5])
print(f'MSE: {nn.MSELoss()(pred, tgt):.4f}')
```

```
MSE: 0.1667
```

nn.L1Loss()

Mean Absolute Error. Robust to outliers.

```
import torch, torch.nn as nn
print(f'L1: {nn.L1Loss()(torch.tensor([1., 5.]), torch.tensor([2., 3.])):.4f}')
```

```
L1: 1.5000
```

nn.SmoothL1Loss() / nn.HuberLoss()

Best of both: MSE for small errors, L1 for large. Used in object detection.

```
import torch, torch.nn as nn
print(f'Huber: {nn.HuberLoss()(torch.tensor([1., 10.]), torch.tensor([2., 3.])):.4f}')
```

```
Huber: 3.5000
```

nn.CrossEntropyLoss()

LogSoftmax + NLLLoss. THE classification loss. Expects raw logits!

```
import torch, torch.nn as nn
logits = torch.tensor([[2., 0.5, 0.1], [0.1, 2., 0.5]])
labels = torch.tensor([0, 1])
print(f'CE: {nn.CrossEntropyLoss()(logits, labels):.4f}')
```

```
CE: 0.3168
```

nn.NLLLoss()

Negative log likelihood. Pair with LogSoftmax.

```
import torch, torch.nn as nn
log_probs = nn.LogSoftmax(dim=1)(torch.tensor([[2., 0.5], [0.5, 2.]]))
print(f'NLL: {nn.NLLLoss()(log_probs, torch.tensor([0, 1])):.4f}')
```

```
NLL: 0.2014
```

nn.BCELoss()

Binary cross entropy. Input MUST be probabilities (0-1)!

```
import torch, torch.nn as nn
print(f'BCE: {nn.BCELoss()(torch.tensor([0.8, 0.4]), torch.tensor([1., 0.])):.4f}')
```

```
BCE: 0.3670
```

nn.BCEWithLogitsLoss()

BCE + built-in Sigmoid. More stable. Use this over BCELoss!

```
import torch, torch.nn as nn
print(f'BCELogits: {nn.BCEWithLogitsLoss()(torch.tensor([1.5, -0.5]), torch.tensor([1., 0.])):.4f}')
```

```
BCELogits: 0.3377
```

nn.KLDivLoss()

KL divergence. Measures how one distribution differs from another.

```
import torch, torch.nn as nn
log_p = nn.LogSoftmax(dim=0)(torch.tensor([0.5, 0.3, 0.2]))
q = nn.Softmax(dim=0)(torch.tensor([0.4, 0.4, 0.2]))
print(f'KL: {nn.KLDivLoss(reduction="sum")(log_p, q):.4f}')
```

```
KL: 0.0035
```

nn.PoissonNLLLoss()

For count data (Poisson regression).

```
import torch, torch.nn as nn
print(f'Poisson: {nn.PoissonNLLLoss()(torch.tensor([1.]), torch.tensor([2.])):.4f}')
```

```
Poisson: 0.7183
```

nn.CosineEmbeddingLoss()

Cosine-similarity based loss for learning embeddings.

```
import torch, torch.nn as nn
x1 = torch.randn(2, 5)
x2 = torch.randn(2, 5)
y = torch.tensor([1, -1])  # similar, dissimilar
print(f'CosEmb: {nn.CosineEmbeddingLoss()(x1, x2, y):.4f}')
```

```
CosEmb: 0.2012
```

nn.TripletMarginLoss()

anchor-positive close, anchor-negative far. Face recognition!

```
import torch, torch.nn as nn
```

```
a = torch.randn(2, 128)
p = a + 0.1 * torch.randn(2, 128)
n_ = torch.randn(2, 128)
print(f'Triplet: {nn.TripletMarginLoss()(a, p, n_):.4f}')
```

```
Triplet: 0.0000
```

nn.TripletMarginWithDistanceLoss()

Triplet loss with custom distance function.

```
import torch, torch.nn as nn
loss = nn.TripletMarginWithDistanceLoss(distance_function=nn.PairwiseDistance())
print(f'Custom dist triplet: {loss(torch.randn(2,5), torch.randn(2,5), torch.randn(2,5)):.4f}')
```

```
Custom dist triplet: 0.0000
```

nn.MarginRankingLoss()

For ranking/ordering tasks.

```
import torch, torch.nn as nn
x1 = torch.tensor([1., 2.])
x2 = torch.tensor([2., 1.])
y = torch.tensor([1., 1.])  # x1 should rank higher
print(f'Ranking: {nn.MarginRankingLoss()(x1, x2, y):.4f}')
```

```
Ranking: 0.5000
```

nn.HingeEmbeddingLoss()

Hinge loss for similarity/dissimilarity learning.

```
import torch, torch.nn as nn
print(f'Hinge: {nn.HingeEmbeddingLoss()(torch.tensor([0.5, 1.5]), torch.tensor([1., -1.])):.4f}')
```

```
Hinge: 0.2500
```

nn.MultiMarginLoss()

Multi-class hinge loss (SVM-style).

```
import torch, torch.nn as nn
print(f'MultiMargin: {nn.MultiMarginLoss()(torch.tensor([[0.1, 0.8, 0.1]]), torch.tensor([1])):.4f}')
```

```
MultiMargin: 0.2000
```

nn.MultiLabelMarginLoss()

Multi-label classification with margin.

```
import torch, torch.nn as nn
print(f'MLMargin: {nn.MultiLabelMarginLoss()(torch.tensor([[0.1, 0.8, 0.1]]), torch.tensor([[1, -1, -1]])):.4f}')
```

```
MLMargin: 0.2000
```

nn.MultiLabelSoftMarginLoss()

Multi-label with soft margin (sigmoid + BCE).

```
import torch, torch.nn as nn
print(f'MLSoft: {nn.MultiLabelSoftMarginLoss()(torch.tensor([[0.1, 0.8]]), torch.tensor([[0., 1.]])):.4f}')
```

```
MLSoft: 0.5577
```

nn.CTCLoss()

Connectionist Temporal Classification. Speech recognition, OCR.

```
import torch, torch.nn as nn
T, C, N = 50, 20, 2
input = torch.randn(T, N, C).log_softmax(2)
target = torch.randint(1, C, (N, 30))
lens = torch.full((N,), T, dtype=torch.long)
tgt_lens = torch.full((N,), 30, dtype=torch.long)
print(f'CTC: {nn.CTCLoss()(input, target, lens, tgt_lens):.4f}')
```

```
CTC: 3.8546
```

nn.GaussianNLLLoss()

For heteroscedastic regression (predicting mean + variance).

```
import torch, torch.nn as nn
mean = torch.tensor([1., 2.])
var = torch.tensor([0.5, 1.])
target = torch.tensor([1.5, 2.5])
print(f'GaussNLL: {nn.GaussianNLLLoss()(mean, target, var):.4f}')
```

```
GaussNLL: 0.0142
```

!LR Schedule Chart

19. Optimizers & Schedulers

25 functions with examples | 25 total in this category

optim.SGD

Stochastic Gradient Descent. The classic. Add momentum for speed.

```
import torch
model = torch.nn.Linear(5, 1)
opt = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
print(f'SGD: lr=0.01, momentum=0.9')
```

```
SGD: lr=0.01, momentum=0.9
```

optim.Adam

Adaptive moment estimation. Default choice. Works great out of the box.

```
import torch
opt = torch.optim.Adam(torch.nn.Linear(5, 1).parameters(), lr=1e-3)
print(f'Adam: lr=0.001 (adaptive lr per parameter)')
```

```
Adam: lr=0.001 (adaptive lr per parameter)
```

optim.AdamW

Adam with CORRECT weight decay. Use this for transformers!

```
import torch
opt = torch.optim.AdamW(torch.nn.Linear(5, 1).parameters(), lr=1e-3, weight_decay=0.01)
print(f'AdamW: decoupled weight decay=0.01')
```

```
AdamW: decoupled weight decay=0.01
```

optim.Adamax

Adam variant using infinity norm. Sometimes better for embeddings.

```
import torch
opt = torch.optim.Adamax(torch.nn.Linear(5, 1).parameters(), lr=2e-3)
print('Adamax: Adam with infinity-norm')
```

```
Adamax: Adam with infinity-norm
```

optim.RMSprop

Good for RNNs. Hinton's optimizer.

```
import torch
opt = torch.optim.RMSprop(torch.nn.Linear(5, 1).parameters(), lr=0.01)
print('RMSprop: adaptive lr using squared gradient average')
```

```
RMSprop: adaptive lr using squared gradient average
```

optim.Adagrad

Good for sparse features. LR decreases over time.

```
import torch
opt = torch.optim.Adagrad(torch.nn.Linear(5, 1).parameters(), lr=0.01)
print('Adagrad: adapts lr per parameter')
```

```
Adagrad: adapts lr per parameter
```

optim.Adadelta

Extension of Adagrad. No learning rate needed.

```
import torch
opt = torch.optim.Adadelta(torch.nn.Linear(5, 1).parameters())
print('Adadelta: no need to set lr!')
```

```
Adadelta: no need to set lr!
```

optim.RAdam

Adam with automatic warmup. More stable early training.

```
import torch
opt = torch.optim.RAdam(torch.nn.Linear(5, 1).parameters(), lr=1e-3)
print('RAdam: rectified Adam (better warmup)')
```

```
RAdam: rectified Adam (better warmup)
```

optim.NAdam

Adam with Nesterov momentum. Often slightly better.

```
import torch
opt = torch.optim.NAdam(torch.nn.Linear(5, 1).parameters(), lr=1e-3)
print('NAdam: Nesterov + Adam')
```

```
NAdam: Nesterov + Adam
```

optim.LBFGS

Quasi-Newton method. Needs a closure function. For small models.

```
import torch
opt = torch.optim.LBFGS(torch.nn.Linear(5, 1).parameters())
print('L-BFGS: second-order optimizer (needs closure)')
```

```
L-BFGS: second-order optimizer (needs closure)
```

optim.SparseAdam

Adam for sparse tensors. Use with sparse embeddings.

```
import torch, torch.nn as nn
emb = nn.Embedding(100, 10, sparse=True)
opt = torch.optim.SparseAdam(emb.parameters())
print('SparseAdam: for sparse gradients (embeddings)')
```

```
SparseAdam: for sparse gradients (embeddings)
```

optim.ASGD

Averaged SGD. Better generalization in theory.

```
import torch
opt = torch.optim.ASGD(torch.nn.Linear(5, 1).parameters())
print('ASGD: averaged SGD')
```

```
ASGD: averaged SGD
```

lr_scheduler.StepLR

Simple step decay. Most common scheduler.

```
import torch
opt = torch.optim.SGD(torch.nn.Linear(5,1).parameters(), lr=0.1)
s = torch.optim.lr_scheduler.StepLR(opt, step_size=10, gamma=0.5)
print(f'Decay lr by 0.5x every 10 epochs')
```

```
Decay lr by 0.5x every 10 epochs
```

lr_scheduler.MultiStepLR

Decay at specific milestones. Common in CV.

```
import torch
opt = torch.optim.SGD(torch.nn.Linear(5,1).parameters(), lr=0.1)
s = torch.optim.lr_scheduler.MultiStepLR(opt, milestones=[30, 80], gamma=0.1)
print('Decay at epochs 30 and 80')
```

```
Decay at epochs 30 and 80
```

lr_scheduler.ExponentialLR

Smooth exponential decay every epoch.

```
import torch
opt = torch.optim.SGD(torch.nn.Linear(5,1).parameters(), lr=0.1)
s = torch.optim.lr_scheduler.ExponentialLR(opt, gamma=0.95)
print('Multiply lr by 0.95 each epoch')
```

```
Multiply lr by 0.95 each epoch
```

lr_scheduler.CosineAnnealingLR

Smooth cosine curve from initial lr to 0. Beautiful and effective.

```
import torch
opt = torch.optim.SGD(torch.nn.Linear(5,1).parameters(), lr=0.1)
s = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=100)
print('Cosine decay over 100 epochs')
```

```
Cosine decay over 100 epochs
```

lr_scheduler.CosineAnnealingWarmRestarts

Cosine decay + periodic restarts. Explore more of loss landscape.

```
import torch
opt = torch.optim.SGD(torch.nn.Linear(5,1).parameters(), lr=0.1)
s = torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(opt, T_0=10)
print('Cosine with warm restarts every 10 epochs')
```

```
Cosine with warm restarts every 10 epochs
```

lr_scheduler.OneCycleLR

Super-convergence policy. Often the best scheduler!

```
import torch
opt = torch.optim.SGD(torch.nn.Linear(5,1).parameters(), lr=0.01)
s = torch.optim.lr_scheduler.OneCycleLR(opt, max_lr=0.1, total_steps=1000)
print('Warmup -> peak -> cosine decay')
```

```
Warmup -> peak -> cosine decay
```

lr_scheduler.CyclicLR

Cycle lr between bounds. Good for finding lr range.

```
import torch
opt = torch.optim.SGD(torch.nn.Linear(5,1).parameters(), lr=0.001)
s = torch.optim.lr_scheduler.CyclicLR(opt, base_lr=0.001, max_lr=0.01, step_size_up=200)
print('Triangular cycling between base and max lr')
```

```
Triangular cycling between base and max lr
```

lr_scheduler.ReduceLROnPlateau

Adaptive! Watches your metric and decays when stuck.

```
import torch
opt = torch.optim.SGD(torch.nn.Linear(5,1).parameters(), lr=0.1)
s = torch.optim.lr_scheduler.ReduceLROnPlateau(opt, patience=5, factor=0.5)
print('Reduce lr when val loss plateaus')
```

```
Reduce lr when val loss plateaus
```

lr_scheduler.LambdaLR

Define your own lr schedule with a lambda/function.

```
import torch
opt = torch.optim.SGD(torch.nn.Linear(5,1).parameters(), lr=0.1)
s = torch.optim.lr_scheduler.LambdaLR(opt, lr_lambda=lambda e: 0.95**e)
print('Custom lr function')
```

```
Custom lr function
```

lr_scheduler.LinearLR

Linear warmup or decay. Often used for warmup phase.

```
import torch
opt = torch.optim.SGD(torch.nn.Linear(5,1).parameters(), lr=0.1)
s = torch.optim.lr_scheduler.LinearLR(opt, start_factor=0.1, total_iters=10)
print('Linear warmup over 10 iters')
```

```
Linear warmup over 10 iters
```

lr_scheduler.PolynomialLR

Polynomial learning rate decay.

```
import torch
opt = torch.optim.SGD(torch.nn.Linear(5,1).parameters(), lr=0.1)
s = torch.optim.lr_scheduler.PolynomialLR(opt, total_iters=100, power=2.0)
print('Polynomial decay')
```

```
Polynomial decay
```

lr_scheduler.SequentialLR

Chain schedulers in sequence. Warmup then decay!

```
import torch
opt = torch.optim.SGD(torch.nn.Linear(5,1).parameters(), lr=0.1)
s1 = torch.optim.lr_scheduler.LinearLR(opt, start_factor=0.1, total_iters=5)
s2 = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=95)
seq = torch.optim.lr_scheduler.SequentialLR(opt, [s1, s2], milestones=[5])
print('LinearLR warmup then CosineAnnealing')
```

```
LinearLR warmup then CosineAnnealing
```

lr_scheduler.ChainedScheduler

Compose schedulers multiplicatively.

```
import torch
opt = torch.optim.SGD(torch.nn.Linear(5,1).parameters(), lr=0.1)
print('ChainedScheduler: multiply multiple schedulers together')
```

```
ChainedScheduler: multiply multiple schedulers together
```

!Normalization Chart

20. Regularization

7 functions with examples | 16 total in this category

nn.Dropout(p)

Randomly zero elements. Active in training only!

```
import torch, torch.nn as nn
torch.manual_seed(42)
d = nn.Dropout(0.5)
x = torch.ones(10)
print(f'Train: {d(x)}')
d.eval()
print(f'Eval:  {d(x)}')
```

```
Train: tensor([2., 2., 2., 2., 0., 2., 0., 0., 2., 2.])
Eval:  tensor([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
```

nn.Dropout2d(p)

Drop entire feature channels (for conv layers).

```
import torch, torch.nn as nn
d = nn.Dropout2d(0.5)
print(f'Drops entire channels: {d(torch.ones(1, 4, 3, 3)).shape}')
```

```
Drops entire channels: torch.Size([1, 4, 3, 3])
```

• nn.Dropout3d(p)

• nn.AlphaDropout(p)

nn.BatchNorm1d(n)

Normalize across batch. Stabilizes training like magic.

```
import torch, torch.nn as nn
bn = nn.BatchNorm1d(3)
x = torch.randn(4, 3) * 10 + 5
out = bn(x)
print(f'Before: mean={x.mean(0).data.tolist()}')
print(f'After:  mean={[f"{v:.2f}" for v in out.mean(0).data.tolist()]}')
```

```
Before: mean=[9.939366340637207, 7.899936199188232, 5.7497944831848145]
After:  mean=['0.00', '0.00', '0.00']
```

nn.BatchNorm2d(n)

BatchNorm for 2D conv layers (N, C, H, W).

```
import torch, torch.nn as nn
print(f'BatchNorm2d: {nn.BatchNorm2d(3)(torch.randn(2, 3, 4, 4)).shape}')
```

```
BatchNorm2d: torch.Size([2, 3, 4, 4])
```

• nn.BatchNorm3d(n)

• nn.SyncBatchNorm

nn.LayerNorm(shape)

Normalize across features. THE norm for transformers.

```
import torch, torch.nn as nn
ln = nn.LayerNorm(4)
x = torch.randn(2, 3, 4)
print(f'LayerNorm: {ln(x).shape}')
```

```
LayerNorm: torch.Size([2, 3, 4])
```

nn.GroupNorm(groups, ch)

Split channels into groups, normalize each. Works with batch_size=1!

```
import torch, torch.nn as nn
print(f'GroupNorm: {nn.GroupNorm(2, 4)(torch.randn(2, 4, 3, 3)).shape}')
```

```
GroupNorm: torch.Size([2, 4, 3, 3])
```

nn.InstanceNorm2d(n)

Normalize per-sample per-channel. Style transfer go-to.

```
import torch, torch.nn as nn
print(f'InstanceNorm: {nn.InstanceNorm2d(3)(torch.randn(2, 3, 4, 4)).shape}')
```

```
InstanceNorm: torch.Size([2, 3, 4, 4])
```

• nn.LocalResponseNorm(size)

• nn.utils.weight_norm(module)

• nn.utils.spectral_norm(module)

• weight_decay (L2)

• label smoothing

!Convolution Chart

21. Convolutions & Pooling

3 functions with examples | 22 total in this category

• nn.Conv1d(in, out, k)

nn.Conv2d(in, out, k)

2D convolution. THE layer for image processing.

```
import torch, torch.nn as nn
conv = nn.Conv2d(3, 16, 3, padding=1)
print(f'Input: (1,3,32,32) -> Output: {conv(torch.randn(1,3,32,32)).shape}')
```

```
Input: (1,3,32,32) -> Output: torch.Size([1, 16, 32, 32])
```

• nn.Conv3d(in, out, k)

• nn.ConvTranspose1d

• nn.ConvTranspose2d

• nn.ConvTranspose3d

• nn.MaxPool1d(k)

nn.MaxPool2d(k)

Downsample by taking max in each window.

```
import torch, torch.nn as nn
pool = nn.MaxPool2d(2)
print(f'MaxPool: (1,1,4,4) -> {pool(torch.randn(1,1,4,4)).shape}')
```

```
MaxPool: (1,1,4,4) -> torch.Size([1, 1, 2, 2])
```

• nn.MaxPool3d(k)

• nn.AvgPool1d(k)

• nn.AvgPool2d(k)

• nn.AvgPool3d(k)

• nn.AdaptiveMaxPool2d(out)

nn.AdaptiveAvgPool2d(out)

Pool to exact output size. The bridge between conv and linear!

```
import torch, torch.nn as nn
pool = nn.AdaptiveAvgPool2d((1, 1))
```

```
print(f'Any size -> (1,1): {pool(torch.randn(1, 64, 7, 7)).shape}')
```

```
Any size -> (1,1): torch.Size([1, 64, 1, 1])
```

• nn.MaxUnpool2d(k)

• nn.LPPool2d(p, k)

• nn.Unfold(k)

• nn.Fold(out, k)

• F.interpolate(x, size, mode)

• F.pad(x, pad, mode)

• F.affine_grid(theta, size)

• F.grid_sample(x, grid)

!Recurrent Layers Chart

22. Recurrent Layers

2 functions with examples | 10 total in this category

• nn.RNN(in, h)

nn.LSTM(in, h)

Long Short-Term Memory. Gates prevent vanishing gradients.

```
import torch, torch.nn as nn
lstm = nn.LSTM(10, 20, batch_first=True)
x = torch.randn(2, 5, 10)  # batch, seq, feat
out, (h, c) = lstm(x)
print(f'Output: {out.shape}, Hidden: {h.shape}')
```

```
Output: torch.Size([2, 5, 20]), Hidden: torch.Size([1, 2, 20])
```

nn.GRU(in, h)

Gated Recurrent Unit. Simpler than LSTM, often just as good.

```
import torch, torch.nn as nn
gru = nn.GRU(10, 20, batch_first=True)
out, h = gru(torch.randn(2, 5, 10))
print(f'Output: {out.shape}')
```

```
Output: torch.Size([2, 5, 20])
```

• nn.RNNCell(in, h)

• nn.LSTMCell(in, h)

• nn.GRUCell(in, h)

• nn.utils.rnn.pack_padded_sequence

• nn.utils.rnn.pad_packed_sequence

• nn.utils.rnn.pad_sequence

• nn.utils.rnn.pack_sequence

Part V: Advanced

!Attention Chart

23. Attention & Transformers

6 functions with examples | 9 total in this category

nn.MultiheadAttention(d, h)

Multi-head attention. The heart of every transformer.

```
import torch, torch.nn as nn
mha = nn.MultiheadAttention(embed_dim=8, num_heads=2, batch_first=True)
x = torch.randn(2, 5, 8)
out, w = mha(x, x, x)
print(f'Self-attention: {x.shape} -> {out.shape}')
```

```
Self-attention: torch.Size([2, 5, 8]) -> torch.Size([2, 5, 8])
```

nn.TransformerEncoderLayer

Attention + FFN + LayerNorm + Residual. One transformer block.

```
import torch, torch.nn as nn
layer = nn.TransformerEncoderLayer(d_model=8, nhead=2, dim_feedforward=32, batch_first=True)
print(f'Encoder layer: {layer(torch.randn(2, 5, 8)).shape}')
```

```
Encoder layer: torch.Size([2, 5, 8])
```

nn.TransformerEncoder

Stack of encoder layers. BERT-style encoding.

```
import torch, torch.nn as nn
layer = nn.TransformerEncoderLayer(d_model=8, nhead=2, batch_first=True)
```

```
enc = nn.TransformerEncoder(layer, num_layers=6)
print(f'6-layer encoder: {enc(torch.randn(2, 5, 8)).shape}')
```

```
6-layer encoder: torch.Size([2, 5, 8])
```

• nn.TransformerDecoderLayer

• nn.TransformerDecoder

• nn.Transformer

F.scaled_dot_product_attention

Flash Attention! Hardware-optimized, fused kernel. Use this!

```
import torch, torch.nn.functional as F
q = k = v = torch.randn(2, 4, 5, 8)  # batch, heads, seq, dim
out = F.scaled_dot_product_attention(q, k, v)
print(f'SDPA: {out.shape}')
```

```
SDPA: torch.Size([2, 4, 5, 8])
```

positional encoding (sinusoidal)

Add position info to embeddings. sin/cos at different frequencies.

```
import torch, math
def pos_enc(seq_len, d):
    pe = torch.zeros(seq_len, d)
    pos = torch.arange(seq_len).unsqueeze(1).float()
    div = torch.exp(torch.arange(0, d, 2).float() * -(math.log(10000) / d))
    pe[:, 0::2] = torch.sin(pos * div)
    pe[:, 1::2] = torch.cos(pos * div)
    return pe
print(f'PE(10, 8):\n{pos_enc(10, 8)[:3, :4]}')
```

```
PE(10, 8):
tensor([[ 0.0000,  1.0000,  0.0000,  1.0000],
        [ 0.8415,  0.5403,  0.0998,  0.9950],
        [ 0.9093, -0.4161,  0.1987,  0.9801]])
```

self-attention from scratch

softmax(QK^T / sqrt(d)) V. That's it. That's the paper.

```
import torch, torch.nn.functional as F, math
d = 4
Q = K = V = torch.randn(2, 3, d)  # batch, seq, dim
scores = Q @ K.transpose(-2, -1) / math.sqrt(d)
weights = F.softmax(scores, dim=-1)
out = weights @ V
print(f'Attention output: {out.shape}')
print(f'Weight row sums: {weights[0].sum(-1)}')
```

```
Attention output: torch.Size([2, 3, 4])
Weight row sums: tensor([1.0000, 1.0000, 1.0000])
```

!DataLoader Chart

24. Data Loading

3 functions with examples | 15 total in this category

Dataset

Custom dataset: __len__ + __getitem__. The standard pattern.

```
import torch
from torch.utils.data import Dataset
class MyDS(Dataset):
    def __init__(self, n): self.data = torch.arange(n).float()
    def __len__(self): return len(self.data)
    def __getitem__(self, i): return self.data[i], self.data[i]**2
ds = MyDS(5)
print(f'len={len(ds)}, ds[3]={ds[3]}')
```

```
len=5, ds[3]=(tensor(3.), tensor(9.))
```

• IterableDataset

• TensorDataset

• ConcatDataset

• Subset

DataLoader

Batches + shuffling + multi-process loading. The training feeder.

```
import torch
from torch.utils.data import DataLoader, TensorDataset
ds = TensorDataset(torch.randn(20, 3), torch.randint(0, 2, (20,)))
loader = DataLoader(ds, batch_size=4, shuffle=True)
x, y = next(iter(loader))
print(f'Batch: x={x.shape}, y={y.shape}')
```

```
Batch: x=torch.Size([4, 3]), y=torch.Size([4])
```

random_split

Split dataset randomly. Train/val/test splits made easy.

```
import torch
from torch.utils.data import TensorDataset, random_split
ds = TensorDataset(torch.randn(100, 5))
train, val = random_split(ds, [80, 20])
print(f'Train: {len(train)}, Val: {len(val)}')
```

```
Train: 80, Val: 20
```

• SubsetRandomSampler

• WeightedRandomSampler

• BatchSampler

• SequentialSampler

• collate_fn (custom)

• pin_memory

• num_workers

• persistent_workers

!Training Curves Chart

25. Training Patterns

2 functions with examples | 9 total in this category

• model.train() / model.eval()

training loop

The sacred loop: forward -> loss -> zero_grad -> backward -> step.

```
import torch, torch.nn as nn
torch.manual_seed(42)
X = torch.randn(100, 5)
y = (X.sum(1) > 0).float().unsqueeze(1)
model = nn.Sequential(nn.Linear(5, 10), nn.ReLU(), nn.Linear(10, 1), nn.Sigmoid())
opt = torch.optim.Adam(model.parameters(), lr=0.01)
for epoch in range(5):
    pred = model(X)
    loss = nn.BCELoss()(pred, y)
    opt.zero_grad()
    loss.backward()
    opt.step()
    acc = ((pred > 0.5) == y).float().mean()
    print(f'Epoch {epoch}: loss={loss:.4f} acc={acc:.2f}')
```

```
Epoch 0: loss=0.6922 acc=0.52
Epoch 1: loss=0.6813 acc=0.57
Epoch 2: loss=0.6704 acc=0.64
Epoch 3: loss=0.6595 acc=0.69
Epoch 4: loss=0.6485 acc=0.77
```

• validation loop

• early stopping

• gradient accumulation

gradient clipping

Prevent exploding gradients. Essential for RNNs and transformers.

```
import torch, torch.nn as nn
model = nn.Linear(5, 1)
loss = model(torch.randn(2, 5) * 100).sum()
loss.backward()
norm = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
print(f'Grad norm: {norm:.2f} -> clipped to 1.0')
```

```
Grad norm: 283.67 -> clipped to 1.0
```

• mixed precision training

• model checkpointing

• distributed training overview

!Hooks Chart

26. Custom Modules & Hooks

0 functions with examples | 9 total in this category

• custom nn.Module

• forward hooks

• backward hooks

• weight initialization (xavier, kaiming, etc.)

• nn.utils.parametrize

• model.register_forward_hook

• model.register_full_backward_hook

• model.register_forward_pre_hook

• torch.nn.init.*

!AMP Chart

27. Mixed Precision (AMP)

1 functions with examples | 5 total in this category

torch.autocast

Auto mixed precision: use lower precision where safe. ~2x speedup!

```
import torch
with torch.autocast(device_type='cpu', dtype=torch.bfloat16):
    out = torch.nn.Linear(5, 3)(torch.randn(2, 5))
    print(f'Autocast dtype: {out.dtype}')
```

```
Autocast dtype: torch.bfloat16
```

• torch.amp.GradScaler

• amp training loop

• bfloat16 training

• torch.set_float32_matmul_precision

!Transforms Chart

28. Transforms (torchvision)

0 functions with examples | 14 total in this category

• transforms.Compose

• transforms.ToTensor()

• transforms.Normalize(mean, std)

• transforms.Resize(size)

• transforms.CenterCrop(size)

• transforms.RandomCrop(size)

• transforms.RandomHorizontalFlip(p)

• transforms.RandomRotation(deg)

• transforms.ColorJitter

• transforms.RandomAffine

• transforms.RandomErasing

• transforms.GaussianBlur(k)

• transforms.RandomResizedCrop(size)

• transforms.v2 (new API)

!JIT Chart

29. JIT & TorchScript

2 functions with examples | 5 total in this category

torch.jit.trace(model, input)

Record operations from example input. Fast inference.

```
import torch
m = torch.nn.Linear(5, 3)
traced = torch.jit.trace(m, torch.randn(1, 5))
print(f'Traced: {traced(torch.randn(2, 5)).shape}')
```

```
Traced: torch.Size([2, 3])
```

torch.jit.script(model)

Compile Python to TorchScript. Handles control flow!

```
import torch
@torch.jit.script
def f(x: torch.Tensor) -> torch.Tensor:
    return x * 2 + 1
print(f(torch.tensor([1., 2., 3.])))
```

```
Error: Could not get qualified name for class 'f': __module__ can't be None.
```

• @torch.jit.script decorator

• torch.jit.save / torch.jit.load

• torch.jit.freeze(model)

!Profiling Chart

30. Profiling & Debugging

2 functions with examples | 6 total in this category

torch.autograd.detect_anomaly()

Find where NaN/Inf first appears in backward pass. Debugging hero!

```
import torch
with torch.autograd.detect_anomaly():
    x = torch.tensor([1.], requires_grad=True)
    y = x ** 2
    y.backward()
    print(f'No anomaly detected. grad={x.grad}')
```

```
No anomaly detected. grad=tensor([2.])
```

• torch.autograd.set_detect_anomaly(True)

• torch.autograd.profiler.profile

• torch.profiler.profile

torch.set_printoptions(...)

Control tensor display: precision, threshold, linewidth, etc.

```
import torch
torch.set_printoptions(precision=2, sci_mode=False)
print(torch.tensor([0.123456789, 1234.5]))
torch.set_printoptions(precision=4)  # reset
```

```
tensor([    0.12,  1234.50])
```

• model summary (torchinfo)

!Distributed Chart

31. Distributed Training

0 functions with examples | 6 total in this category

• torch.nn.DataParallel

• torch.nn.parallel.DistributedDataParallel

• torch.distributed.init_process_group

• torch.distributed.DistributedSampler

• torch.distributed.all_reduce

• torch.distributed.broadcast
